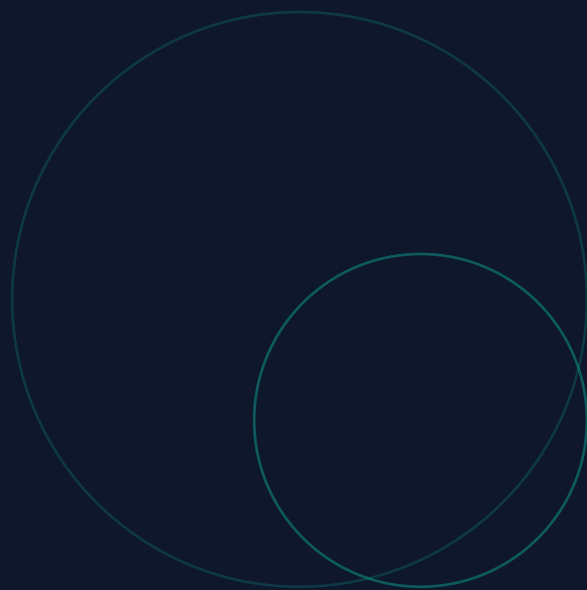


O Grande Livro Visual da Auditoria AIDD

A Fábrica de Software Perfeita: Uma Viagem Pelas 8 Ferramentas Macro, 68 Micro-Ferramentas, 50 Guardas e 141 Componentes

Do macro ao micro: o ecossistema como um todo, os três fluxos canônicos de criação e cada uma das oito ferramentas — arquitetura, engenharia agêntica e economia de tokens, camada por camada.



Equipe de Engenharia Canônica do Ecossistema AIDD e Antigravity Agent

24 de setembro de 2026

SOBRE ESTA OBRA

Este livro é a documentação visual e pedagógica definitiva de cada engrenagem do Ecossistema AIDD. Escrito através da técnica dual (Na Festa para explicar de forma lúdica que até uma criança entenda, e Na Casa para fornecer o rigor técnico, os caminhos absolutos dos arquivos no disco e as 11 dimensões milimetricamente auditadas). Cobre com precisão cirúrgica as 8 Ferramentas Macro, as 68 Micro-Ferramentas (Skills), os 50 Quality Gates determinísticos e os 141 Componentes Agnósticos sincronizados em múltiplos ambientes.

Sumário

Prólogo: Bem-vindo à Cidade da Fábrica de Brinquedos Perfeita	6
PARTE I: AS 8 FERRAMENTAS MACRO (AS GRANDES OFICINAS)	7
Capítulo 1: AIDD Forge — O Altar da Fundação e os Guardiões das Leis	7
Na Festa (A Metáfora Memorável)	7
Na Casa (A Foto Técnica Rigorosa e as 11 Dimensões)	7
Ficha de Auditoria Bit a Bit: aidd-forge	8
SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)	8
1. Matriz Contratual das 11 Dimensões	8
2. Evidência de Testes e Portões	9
3. Diagnóstico de Não-Conformidades	9
SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA	9
SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)	9
Capítulo 2: AIDD Planner — A Mesa do Arquiteto e o Livro de Receitas	9
Na Festa (A Metáfora Memorável)	9
Na Casa (A Foto Técnica Rigorosa e as 11 Dimensões)	9
Ficha de Auditoria Bit a Bit: aidd-planner	10
SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)	10
1. Matriz Contratual das 11 Dimensões	10
2. Evidência de Testes e Portões	11
3. Diagnóstico de Não-Conformidades	11
SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA	11
SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)	11
Capítulo 3: AIDD Generator — O Trem Autônomo de 8 Vagões (Fluxo 01: Do Zero Puro)	11
Na Festa (A Metáfora Memorável)	11
Na Casa (A Foto Técnica Rigorosa e as 11 Dimensões)	11
Ficha de Auditoria Bit a Bit: aidd-generator (Fluxo 01 — Do Zero Puro)	12
SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)	12
1. Matriz Contratual das 11 Dimensões	12
2. Evidência de Testes e Portões	13
3. Diagnóstico de Não-Conformidades	13
SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA	13
SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)	13
Capítulo 4: AIDD Factory — A Linha de Montagem de Peças Prontas (Fluxo 02: Open-Source)	13
Na Festa (A Metáfora Memorável)	13
Na Casa (A Foto Técnica Rigorosa e as 11 Dimensões)	13
Ficha de Auditoria Bit a Bit: aidd-factory (Fluxo 02 — Motores Open-Source)	14
SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)	14
1. Matriz Contratual das 11 Dimensões	14
2. Evidência de Testes e Portões	15
3. Diagnóstico de Não-Conformidades	15
SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA	15
SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)	15
Capítulo 5: AIDD Bridge — A Ponte da Libertação (Fluxo 03: Desacoplamento Low-Code)	15
Na Festa (A Metáfora Memorável)	15
Na Casa (A Foto Técnica Rigorosa e as 11 Dimensões)	16
Ficha de Auditoria Bit a Bit: aidd-bridge (Fluxo 03 — Low-Code / Desacoplamento)	17
SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)	17

1. Matriz Contratual das 11 Dimensões	17
2. Evidência de Testes e Portões	18
3. Diagnóstico de Não-Conformidades	18
SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA	18
SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)	18
Capítulo 6: AIDD Master — O Maestro da Harmonização Monolítica VSA	18
Na Festa (A Metáfora Memorável)	18
Na Casa (A Foto Técnica Rigorosa e as 11 Dimensões)	18
Ficha de Auditoria Bit a Bit: aidd-master (Meso-Camada de Convergência Monolítica VSA)	19
SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)	19
1. Matriz Contratual das 11 Dimensões	19
2. Evidência de Testes e Portões	20
3. Diagnóstico de Não-Conformidades	20
SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA	20
SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)	20
Capítulo 7: AIDD Enterprise — O Cofre de Alta Segurança e Selos Criptográficos	20
Na Festa (A Metáfora Memorável)	20
Na Casa (A Foto Técnica Rigorosa e as 11 Dimensões)	20
Ficha de Auditoria Bit a Bit: aidd-enterprise (Resiliência Crítica SHA-256)	22
SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)	22
1. Matriz Contratual das 11 Dimensões	22
2. Evidência de Testes e Portões	23
3. Diagnóstico de Não-Conformidades	23
SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA	23
SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)	23
Capítulo 8: AIDD Ops — A Usina de Força e a Torre de Vigilância	23
Na Festa (A Metáfora Memorável)	23
Na Casa (A Foto Técnica Rigorosa e as 11 Dimensões)	23
Ficha de Auditoria Bit a Bit: aidd-ops (Infraestrutura, VPS e Observabilidade)	24
SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)	24
1. Matriz Contratual das 11 Dimensões	24
2. Evidência de Testes e Portões	25
3. Diagnóstico de Não-Conformidades	25
SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA	25
SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)	25
PARTE II: O CINTO DE UTILIDADES (AS 68 MICRO-FERRAMENTAS / SKILLS)	26
Na Festa	26
Na Casa (O Catálogo Completo das 68 Skills Canônicas)	26
1. Micro-Ferramenta: agents-sdk	26
2. Micro-Ferramenta: aidd-auditor-4f-runner	26
3. Micro-Ferramenta: aidd-bridge	27
4. Micro-Ferramenta: aidd-bridge-runner	27
5. Micro-Ferramenta: aidd-componentes	28
6. Micro-Ferramenta: aidd-dependencias	28
7. Micro-Ferramenta: aidd-diagnose	28
8. Micro-Ferramenta: aidd-dispatch-runner	29
9. Micro-Ferramenta: aidd-enterprise-runner	29
10. Micro-Ferramenta: aidd-evolucao-runner	30
11. Micro-Ferramenta: aidd-factory-runner	30
12. Micro-Ferramenta: aidd-forge-runner	30

13. Micro-Ferramenta: aidd-freedom	31
14. Micro-Ferramenta: aidd-generator-runner	31
15. Micro-Ferramenta: aidd-grill	32
16. Micro-Ferramenta: aidd-grill-docs	32
17. Micro-Ferramenta: aidd-handoff	33
18. Micro-Ferramenta: aidd-livro-texto	33
19. Micro-Ferramenta: aidd-master-runner	33
20. Micro-Ferramenta: aidd-mcp	34
21. Micro-Ferramenta: aidd-melhoria	34
22. Micro-Ferramenta: aidd-open	35
23. Micro-Ferramenta: aidd-ops-runner	35
24. Micro-Ferramenta: aidd-orca	36
25. Micro-Ferramenta: aidd-orchestrate	36
26. Micro-Ferramenta: aidd-orchestrator-runner	36
27. Micro-Ferramenta: aidd-pipeline-runner	37
28. Micro-Ferramenta: aidd-plan	37
29. Micro-Ferramenta: aidd-planner-runner	38
30. Micro-Ferramenta: aidd-planos	38
31. Micro-Ferramenta: aidd-pure	38
32. Micro-Ferramenta: aidd-skills	39
33. Micro-Ferramenta: aidd-spec	39
34. Micro-Ferramenta: aidd-tdd	40
35. Micro-Ferramenta: aidd-tickets	40
36. Micro-Ferramenta: cloudflare	41
37. Micro-Ferramenta: cloudflare-email-service	41
38. Micro-Ferramenta: cloudflare-one	41
39. Micro-Ferramenta: cloudflare-one-migrations	42
40. Micro-Ferramenta: componentes-runner	42
41. Micro-Ferramenta: debug-issue	43
42. Micro-Ferramenta: dependencia-runner	43
43. Micro-Ferramenta: durable-objects	44
44. Micro-Ferramenta: explore-codebase	44
45. Micro-Ferramenta: fluxo-01-runner	44
46. Micro-Ferramenta: fluxo-02-runner	45
47. Micro-Ferramenta: fluxo-03-runner	45
48. Micro-Ferramenta: freedom	46
49. Micro-Ferramenta: impeccable	46
50. Micro-Ferramenta: mcp-creator-runner	47
51. Micro-Ferramenta: melhoria	47
52. Micro-Ferramenta: nextjs-on-cloudflare	47
53. Micro-Ferramenta: open	48
54. Micro-Ferramenta: orca-plan-orchestrator	48
55. Micro-Ferramenta: orchestrate	49
56. Micro-Ferramenta: plan	49
57. Micro-Ferramenta: planos-auditoria-runner	50
58. Micro-Ferramenta: pure	50
59. Micro-Ferramenta: refactor-safely	50
60. Micro-Ferramenta: review-changes	51

61. Micro-Ferramenta: sandbox-migrate-to-next	51
62. Micro-Ferramenta: sandbox-next	52
63. Micro-Ferramenta: sandbox-stable	52
64. Micro-Ferramenta: skill-creator-runner	53
65. Micro-Ferramenta: turnstile-spin	53
66. Micro-Ferramenta: web-perf	53
67. Micro-Ferramenta: workers-best-practices	54
68. Micro-Ferramenta: wrangler	54
PARTE III: OS 50 GUARDIÕES INCORRÚPTÍVEIS (QUALITY GATES)	56
Na Festa	56
Na Casa (A Matriz dos 50 Quality Gates Canônicos)	56
1. Guarda Incorrúptível: G_amelhoria.py	56
2. Guarda Incorrúptível: G_ANT_LOCKIN_LEGADO.py	56
3. Guarda Incorrúptível: G_ARQUITETURA_DELIVERABLE.py	57
4. Guarda Incorrúptível: G_CLI_HELP_CONSISTENCIA.py	57
5. Guarda Incorrúptível: G_COMPONENTE_AGNOSTICO.py	57
6. Guarda Incorrúptível: G_CONTRACT_ROT.py	58
7. Guarda Incorrúptível: G_DEPENDENCIAS_PIN_HASH.py	58
8. Guarda Incorrúptível: G_DETERMINISMO_LEI_1.py	59
9. Guarda Incorrúptível: G_DISCIPLINA_TESTE_FERRAMENTA.py	59
10. Guarda Incorrúptível: G_DISPATCH_PIPELINE_VSA.py	59
11. Guarda Incorrúptível: G_DOCS_ROT.py	60
12. Guarda Incorrúptível: G_DRIFT_ANALYZER.py	60
13. Guarda Incorrúptível: G_DRIFT_NUCLEO_COMPARTILHADO.py	61
14. Guarda Incorrúptível: G_ECOSSISTEMA_INTEGRIDADE.py	61
15. Guarda Incorrúptível: G_ENV_ROT.py	61
16. Guarda Incorrúptível: G_ESCRITOR_ATOMICO.py	62
17. Guarda Incorrúptível: G ESTRUTURA_ESTADO.py	62
18. Guarda Incorrúptível: G_FRONTEND_LAYERS.py	63
19. Guarda Incorrúptível: G_HADOLINT.py	63
20. Guarda Incorrúptível: G_HANDOFF_MELHORIA.py	63
21. Guarda Incorrúptível: G_HARNESS_COMPAT.py	64
22. Guarda Incorrúptível: G_HONESTIDADE_ROTULO.py	64
23. Guarda Incorrúptível: G_IDIOMA_LEI_4.py	65
24. Guarda Incorrúptível: G_INFRA_COMPOSE.py	65
25. Guarda Incorrúptível: G_ISOLATION_AUDIT.py	65
26. Guarda Incorrúptível: G_LAYOUT_ENTREGA.py	66
27. Guarda Incorrúptível: G_LEI_DECLARA_PORTAO.py	66
28. Guarda Incorrúptível: G_LIVRO_EVIDENCIA.py	67
29. Guarda Incorrúptível: G_LLM_PROMPT_SHIELD.py	67
30. Guarda Incorrúptível: G_MIGRATION_ROT.py	67
31. Guarda Incorrúptível: G_ORQUESTRADOR_SINCRONO.py	68
32. Guarda Incorrúptível: G_PACOTE_CORE.py	68
33. Guarda Incorrúptível: G_PIPELINE_HANDOFF.py	69
34. Guarda Incorrúptível: G_PORTAO_PROVA_QUE_MORDE.py	69
35. Guarda Incorrúptível: G_PROTOCOL_FALLBACK.py	69
36. Guarda Incorrúptível: G_PROTOTYPE_REWRITE.py	70
37. Guarda Incorrúptível: G_QUARTETO_SINE_QUA_NON.py	70
38. Guarda Incorrúptível: G_RESUMO_USUARIO.py	71

39. Guarda Incorruptível: G_SAIDA_BINARIA.py	71
40. Guarda Incorruptível: G_SEGREDOS.py	71
41. Guarda Incorruptível: G_SKILL_ROT.py	72
42. Guarda Incorruptível: G_STACK_PADRAO_OURO.py	72
43. Guarda Incorruptível: G_SUPPLY_CHAIN.py	73
44. Guarda Incorruptível: G_SYNC_CMD_ROT.py	73
45. Guarda Incorruptível: G_TEMPLATE_FORGE_ROT.py	73
46. Guarda Incorruptível: G_TESTES_REAIS.py	74
47. Guarda Incorruptível: G_TRANSACTION_LOG_LRU.py	74
48. Guarda Incorruptível: G_UNIVERSAL_HARNESS.py	75
49. Guarda Incorruptível: G_USER_FACING_PTBR.py	75
50. Guarda Incorruptível: G_ZERO_HEADLESS.py	75
PARTE IV: OS 141 COMPONENTES AGNÓSTICOS (PEÇAS UNIVERSAIS LEGO)	77
Na Festa	77
Na Casa (A Sincronização dos 141 Componentes)	77
As 6 Famílias dos 141 Componentes:	77
Errata das Fichas de 2026-09-22	78
Epílogo: A Orquestra Perfeita	79

Prólogo: Bem-vindo à Cidade da Fábrica de Brinquedos Perfeita

Imagine uma cidade onde existe uma fábrica de brinquedos mágicos. Mas não é uma fábrica qualquer onde as coisas quebram e ninguém sabe o motivo. É uma fábrica que constrói castelos digitais inteiros em poucos minutos!

Para que tudo funcione sem nenhum fio solto, a fábrica possui: 1. **8 Grandes Oficinas Mestras (As Ferramentas Macro)**: Cada uma com um mestre construtor responsável por uma grande missão. 2. **68 Ferramentas de Precisão no Cinto dos Mestres (As Micro-Ferramentas / Skills)**: Lupas, réguas, tesouras mágicas e parafusadeiras que realizam tarefas hiperespecíficas. 3. **50 Guardas Incorrutíveis na Porta (Os Quality Gates)**: Inspetores robóticos que não deixam passar nada se tiver um único parafuso torto ou se faltar a etiqueta de segurança. 4. **141 Peças Fundamentais (Os Componentes Agnósticos)**: Blocos de encaixe perfeito que funcionam em qualquer mesa de trabalho (seja no Claude, no Gemini, no Cursor, no OpenCode ou no Mimo).

Neste livro, nós vamos passear por cada canto dessa fábrica e tirar uma **FOTO** detalhada de cada peça!

PARTE I: AS 8 FERRAMENTAS MACRO (AS GRANDES OFICINAS)

Capítulo 1: AIDD Forge — O Altar da Fundação e os Guardiões das Leis

Na Festa (A Metáfora Memorável)

O Mestre Ferreiro que prepara o chão da oficina, coloca a bigorna, estende as regras na parede e tranca a porta para ninguém entrar bagunçando.

Antes de qualquer parede ser erguida, é este ferreiro que crava as estacas no chão e escreve, na própria parede da oficina, as regras que toda construção futura vai ter que obedecer. Sem terreno preparado e sem regra escrita, não existe planta nem construção — é por isso que ele entra primeiro, e é por isso que ele mesmo não ergue tijolo nenhum.

Na Casa (A Foto Técnica Rigorosa e as 11 Dimensões)

- **Caminho Físico no Disco:** <tools/aidd-forge>
- **Arquivo Canônico de Regras:** <tools/aidd-forge/AGENTS.md>

Ficha de Auditoria Bit a Bit: [aidd-forge](#)

Ferramenta: [tools/aidd-forge](#)
Data da Auditoria: 2026-09-22
Status: AUDITADO / ÍNTEGRO
Responsável: Antigravity / Eco-AIDD Agent

SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)

1. Matriz Contratual das 11 Dimensões

Dimensão	Detalhamento Técnico no Código
1. RECEBE (Input)	CLI Click (<code>init</code> , <code>inject</code> , <code>audit</code> , <code>conform</code>); payloads validados por <code>injection_request.schema.json</code> ; caminho do projeto alvo (<code>path: str</code>).
2. CRIA / PROCESSA	<code>Injector</code> (árvore estrutural), <code>PhaseFencer</code> (10 fases isoladas), <code>SlashRouter</code> (roteador de slash commands agnósticos), <code>UniversalInjector</code> + <code>materializador.py</code> (injeção atômica com rollback transacional), <code>ConformEngine</code> (autocura).
3. ENTREGA (Output)	Árvore de governança (<code>.agent/</code> , <code>.claudes/</code> , <code>.gemini/</code> , <code>.cursor/</code> , <code>.windsurf/</code>), scripts de qualidade em <code>gates/</code> , relatórios auditáveis (Markdown, JSON, HTML) e hook <code>.git/hooks/pre-commit</code> .
4. CONFIGURAÇÕES (Configs)	Flags CLI <code>--force</code> , <code>--dry-run</code> , <code>--format json md html</code> , <code>--output</code> , <code>--item</code> ; mapeamento de aliases de IDE em <code>IDE_RULE_ALIASES</code> .
5. GUARDAS (Gates)	12 quality gates em <code>templates/gates/</code> : <code>G_BLOQUEAR_SEGREDOS</code> , <code>G_CONTRACTS</code> , <code>G_CYBERSECURITY_OWASP</code> , <code>G_DETERMINISMO_LEI_1</code> , <code>G ESTRUTURA_AST</code> , <code>G_HARNESS_COMPAT</code> , <code>G_INJECT</code> , <code>G_PERFORMANCE</code> , <code>G_QUARTETO_SINE_QUA_NON</code> , <code>G_SAIDA_BINARIA</code> , <code>G_STACK_PADRAO_OURO</code> , <code>G_TESTES_REAIS</code> .
6. SCRIPTS DETERMINÍSTICOS (0 LLM)	100% de <code>aidd-forge/core/</code> implementado com AST Python, regex compilado, manipulação atômica de disco e padrão <code>Result Monad</code> (<code>Result.ok</code> , <code>Result.fail</code>).
7. CAMPAINHAS DE ALERTA (Hooks)	<code>GitHooksInstaller</code> instala <code>.git/hooks/pre-commit</code> com execução compulsória dos gates antes de qualquer commit git.
8. PESSOAS / PERSONAS (Agents)	<code>subagent_purger.py</code> garante que subagentes cognitivos sejam estritamente efêmeros e expurgados após validação.
9. TAREFAS ÚNICAS (Skills)	10 skills embutidas: <code>aidd-grill</code> , <code>aidd-spec</code> , <code>aidd-tdd</code> , <code>aidd-tickets</code> , <code>caveman-ultra</code> , <code>cybersecurity-first</code> , <code>impeccable-ui</code> , <code>open-code-review-graph</code> , <code>orca-orchestrator</code> , <code>post-mortem</code> . Skill de orquestração externa: <code>aidd-forge-runner</code> .

Dimensão	Detalhamento Técnico no Código
10. TELEFONES EXTERNOS (MCPs)	Ponto de integração nativo com code-review-graph e suporte a injeção declarativa via forge inject mcp .
11. BILHETES (Rules / AGENTS.md)	Diretrizes em tools/aidd-forge/AGENTS.md : Zero Stubs, Result Monad compulsório, Context-Purge Isolation e Universal Injector.

2. Evidência de Testes e Portões

- **Comando executado:** [pytest tools/aidd-forge/tests](#)
- **Resultado:** 294 testes passaram, 0 falhas, 1 pulado (tempo de execução: 26.84s).
- **Invariantes testadas:** Rollback em falha de escrita, ancoragem em [AGENTS.md](#), isolamento de harnesses, detecção de camadas arquiteturais.

3. Diagnóstico de Não-Conformidades

- Nenhum bug funcional impeditivo encontrado no núcleo do [aidd-forge](#).
- Observação: Garantir que novos quality gates adicionados ao ecossistema raiz sejam sincronizados dinamicamente na pasta [templates/gates/](#) do [aidd-forge](#).

SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA

(Nenhum ticket impeditivo aberto para esta ferramenta no momento. Estado funcional 100% íntegro).

SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)

- **Status do Módulo:** APROVADO & CERTIFICADO
- **Nível de Autonomia:** 100% Determinístico (0 LLM no núcleo)
- **Handoff:** Registrado no [manifesto_auditoria.json](#). Próximo módulo liberado: [aidd-planner](#).

Capítulo 2: AIDD Planner — A Mesa do Arquiteto e o Livro de Receitas

Na Festa (A Metáfora Memorável)

O Grande Arquiteto que escuta o sonho do cliente, desenha a planta baixa em papel milimetrado com todas as medidas, cores e quartos, e entrega uma receita que qualquer cozinheiro consegue seguir.

Com o terreno pronto e as regras na parede, é aqui que o sonho do cliente vira desenho técnico: cada cômodo, medida, porta e contrato descritos em detalhe antes de qualquer prego ser batido. O arquiteto entrega a planta — quem constrói é o próximo mestre da linha.

Na Casa (A Foto Técnica Rigorosa e as 11 Dimensões)

- **Caminho Físico no Disco:** [tools/aidd-planner](#)
- **Arquivo Canônico de Regras:** [tools/aidd-planner/AGENTS.md](#)

Ficha de Auditoria Bit a Bit: aidd-planner

Ferramenta: tools/aidd-planner

Data da Auditoria: 2026-09-22

Status: AUDITADO / ÍNTEGRO

Responsável: Antigravity / Eco-AIDD Agent

SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)

1. Matriz Contratual das 11 Dimensões

Dimensão	Detalhamento Técnico no Código
1. RECEBE (Input)	CLI argparse (init, validate, export, audit). Parâmetros: --fluxo <1\ 2\ 3>, --nome, --slug, --descricao, --dominio, --pasta, --arquivo, --formato <factory\ pipeline\ dispatch>, --saida.
2. CRIA / PROCESSA	planner_engine.py: gerar_template_plano, validar_plano (JSON Schema + regras semânticas), exportar_para_fluxo_factory, exportar_para_pipeline_execucao, compilar_grafo_topologico_vsa. design_system.py: geração determinística (hash do projeto) de paleta cromática e identidade visual única.
3. ENTREGA (Output)	Arquivos: PLANNER.json, DESIGN-SYSTEM.json, HANDOFF_PLANNER_ENGINE.json, artefatos exportados para pipeline/dispatch.
4. CONFIGURAÇÕES (Configs)	Mapeamento MAPA_FLUXOS suportando aliases numéricos e textuais (1, 2, 3, pure, open, freedom). Flags --force, --dry-run.
5. GUARDAS (Gates)	Validação integrada contra schemas/planner_schema.json, verificação obrigatória do Quarteto Sine Qua Non (/docs, /webhooks, /mcp, /docs/guia), BDD Gherkin obrigatório, detecção de ciclos em grafo de dependências VSA.
6. SCRIPTS DETERMINÍSTICOS (0 LLM)	100% determinístico. Cálculos de hash para paleta cromática, validação estrita JSON Schema, algoritmo topológico de Kahn para ordenação DAG de fatias VSA.
7. CAMPAINHAS DE ALERTA (Hooks)	Emite códigos de saída binários (exit 0 sucesso, exit 1 falha) e relatórios formatados em stderr para interceptação por orquestradores superiores.
8. PESSOAS / PERSONAS (Agents)	Atua como motor de intake alimentando personas dos geradores especializados (aidd-generator, aidd-factory, aidd-bridge).
9. TAREFAS ÚNICAS (Skills)	Skills associadas no ecossistema: aidd-planner-runner, aidd-plan, aidd-planos.
10. TELEFONES EXTERNOS (MCPs)	Totalmente desacoplado e autônomo (não necessita de MCP externo para execução de intake e exportação).

Dimensão	Detalhamento Técnico no Código
11. BILHETES (Rules / AGENTS.md)	tools/aidd-planner/AGENTS.md fixa: Invariante Zero Stubs, Quarteto Sine Qua Non ativado por default, envelope estrito aidd-ops para export factory.

2. Evidência de Testes e Portões

- **Comando executado:** `pytest tools/aidd-planner/tests`
- **Resultado:** 24 testes passaram, 0 falhas (tempo de execução: 1.25s).
- **Invariantes testadas:** Rejeição de plano sem quarteto, rejeição de plano sem BDD, exportação para factory envelope ops, compilação de grafo VSA com detecção de ciclo.

3. Diagnóstico de Não-Conformidades

- Nenhum bug impeditivo. Conformidade 100% comprovada na geração de envelopes de handoff formal e compatibilidade com VSA.

SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA

(Nenhum ticket impeditivo aberto. Módulo plenamente funcional).

SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)

- **Status do Módulo:** APROVADO & CERTIFICADO
- **Nível de Autonomia:** 100% Determinístico
- **Handoff:** Registrado no [manifesto_auditoria.json](#). Próximo módulo: [aidd-generator](#) (Fluxo 01).

Capítulo 3: AIDD Generator — O Trem Autônomo de 8 Vagões (Fluxo 01: Do Zero Puro)

Na Festa (A Metáfora Memorável)

Um trem mágico com 8 vagões sequenciais. Ele recebe a ideia pura em uma ponta e, vagão por vagão (pesquisa, desenho, teste, programação), entrega uma cidade inteira de brinquedo montada e funcionando.

Com a planta em mãos, este é o motor que efetivamente ergue a construção do zero: vagão por vagão, a ideia pura vira aplicação funcionando, sem depender de nenhum tijolo pré-fabricado de terceiros.

Na Casa (A Foto Técnica Rigorosa e as 11 Dimensões)

- **Caminho Físico no Disco:** [tools/aidd-generator](#)
- **Arquivo Canônico de Regras:** [tools/aidd-generator/AGENTS.md](#)

Ficha de Auditoria Bit a Bit: aidd-generator (Fluxo 01 — Do Zero Puro)

Ferramenta: `tools/aidd-generator`
Data da Auditoria: 2026-09-22
Status: AUDITADO / ÍNTEGRO
Responsável: Antigravity / Eco-AIDD Agent

SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)

1. Matriz Contratual das 11 Dimensões

Dimensão	Detalhamento Técnico no Código
1. RECEBE (Input)	Ideia textual ou plano via CLI (<code>python scripts/pipeline_completo.py "<ideia>"</code>), flags <code>--modo <eco fast full></code> , <code>--verbose</code> , <code>--skip-fase <N></code> , <code>--retomar</code> , <code>--plano <path.json></code> . Consome contrato de handoff do planner (<code>HANDOFF_PLANNER_ENGINE.json</code>).
2. CRIA / PROCESSA	Pipeline autônoma de 8 fases sequenciais orquestrada por Máquina de Estados Finitos (<code>fsm_engine.py</code>): Fase 1 (Pesquisa), Fase 2 (Análise de Requisitos), Fase 3 (Design Arquitetural & UI), Fase 4 (Planejador de Slices), Fase 5 (Criador de Código/TDD Red), Fase 6 (Documentador & OpenAPI), Fase 7 (Auto-Crítica & Auditoria), Fase 8 (Implementador Green & Refactor).
3. ENTREGA (Output)	Código-fonte completo do projeto (Frontend Next.js + TS + Tailwind, Backend Python puro VSA + SQLite WAL), <code>PLANO-EXECUCAO-ESTRUTURADO.json</code> , testes automatizados, relatórios de auditoria e tokenomics (<code>benchmark_tokenomics.py</code>).
4. CONFIGURAÇÕES (Configs)	Modos de operação: <code>eco</code> (mínimo de tokens), <code>fast</code> (direto ao ponto), <code>full</code> (máxima profundidade). Configuração de orquestração via Prefect (<code>pipeline_prefect.py</code>) ou fallback nativo Python.
5. GUARDAS (Gates)	Validação de transição entre cada uma das 8 fases em <code>scripts/gates/</code> e <code>verificar_gates.py</code> . Rejeição binária imediata se uma fase não atingir 100% dos requisitos contratuais.
6. SCRIPTS DETERMINÍSTICOS (O LLM)	Mecanismo FSM (<code>fsm_engine.py</code>), validação de esquemas de artefatos por JSON Schema Draft 2020-12, checagem de integridade de arquivos gerados e telemetria determinística de tokens.
7. CAMPAINHAS DE ALERTA (Hooks)	Emissão de alertas de quebra de contrato de fase e interrupção compulsória com preservação de estado transacional.
8. PESSOAS / PERSONAS (Agents)	8 personas operacionais especializadas em cada fase (Pesquisador, Analista, Designer, Planejador, Criador, Documentador, Crítico, Implementador).

Dimensão	Detalhamento Técnico no Código
9. TAREFAS ÚNICAS (Skills)	Skills integradas: aidd-generator-runner , aidd-pure , fluxo-01-runner , aidd-spec , aidd-tickets , aidd-tdd .
10. TELEFONES EXTERNOS (MCPs)	Suporte a chamadas de inspeção estática via code-review-graph e preflight checks de ambiente.
11. BILHETES (Rules / AGENTS.md)	tools/aidd-generator/AGENTS.md : Persistência de estado exclusivamente em JSON, transições mecânicas estritas, Zero Stubs na Fase 8.

2. Evidência de Testes e Portões

- **Comando executado:** `pytest tools/aidd-generator/tests`
- **Resultado:** 1006 testes passaram, 0 falhas, 5 pulados (Prefect opcional) em 26.50s.
- **Invariantes testadas:** Transparência de tokens, fencer de isolamento de fases, microtarefas da Fase 8, roteamento de slash commands e validação de gates.

3. Diagnóstico de Não-Conformidades

- Nenhum bug impeditivo. 100% de integridade comprovada nos 1006 testes unitários e de integração.

SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA

(Nenhum ticket impeditivo aberto. Módulo plenamente funcional).

SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)

- **Status do Módulo:** APROVADO & CERTIFICADO
- **Nível de Autonomia:** Pipeline autônoma de 8 fases com barreira determinística.
- **Handoff:** Registrado no [manifesto_auditoria.json](#). Próximo módulo: [aidd-factory](#) (Fluxo 02).

Capítulo 4: AIDD Factory — A Linha de Montagem de Peças Prontas (Fluxo 02: Open-Source)

Na Festa (A Metáfora Memorável)

O Mestre Montador que pega os melhores motores de brinquedo já inventados no mundo (motores abertos) e os conecta perfeitamente com cabos fortes para criar um veículo superpotente sem reinventar a roda.

Também constrói a partir da mesma planta, mas em vez de erguer tijolo por tijolo, monta a casa com módulos open-source já prontos, testados por milhares de outras obras e encaixados sob medida no projeto.

Na Casa (A Foto Técnica Rigorosa e as 11 Dimensões)

- **Caminho Físico no Disco:** `tools/aidd-factory`
- **Arquivo Canônico de Regras:** `tools/aidd-factory/AGENTS.md`

Ficha de Auditoria Bit a Bit: [aidd-factory](#) (Fluxo 02 — Motores Open-Source)

Ferramenta: [tools/aidd-factory](#)
Data da Auditoria: 2026-09-22
Status: AUDITADO / ÍNTEGRO
Responsável: Antigravity / Eco-AIDD Agent

SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)

1. Matriz Contratual das 11 Dimensões

Dimensão	Detalhamento Técnico no Código
1. RECEBE (Input)	Consome estritamente PLANO-INFRAESTRUTURA.json em conformidade com componentes/compartilhado/specs/plano-infraestrutura.schema.json gerado pelo intake do aidd-planner ou aidd-ops . CLI: <code>python scripts/pipeline_factory.py --plano <path.json> --destino <dir></code> .
2. CRIA / PROCESSA	Pipeline de integração de serviços open-source em fatias VSA: Fase 1 (01_analisador.py — análise de nicho e desdobramento em blocos), Fase 4 (04_compose.py — geração determinística de docker-compose.yml), Fase 5 (05_init_db.py — scripts SQL de inicialização multi-tenant/bancos lógicos), Fase 6 (06_env.py — variáveis de ambiente seguras com fallbacks tipados), Fase 9 (09_integracao.py — gateway de integração VSA e Quarteto Sine Qua Non).
3. ENTREGA (Output)	Contrato canônico FACTORY_OUTPUT.json , manifesto de serviços, compose funcional, scripts de banco de dados, gateway FastAPI VSA e stack frontend Next.js + TS + Tailwind.
4. CONFIGURAÇÕES (Configs)	Parâmetros de CLI (<code>--plano</code> , <code>--destino</code> , <code>--dry-run</code> , <code>--skip-frontend</code>). Suporte a nichos de catálogo fixo (clinicas , delivery , farmacias , b2b_industrial , energia_solar) e nicho dinâmico (<code>dinamico_<slug></code>).
5. GUARDAS (Gates)	<code>G_FACTORY_ANALYSIS</code> , <code>G_FACTORY_COMPOSE</code> , <code>G_FACTORY_ENV</code> , <code>G_FACTORY_INIT_DB</code> , <code>G_FACTORY_INTEGRATION</code> e <code>G_FACTORY_MVP</code> (em <code>tools/aidd-factory/gates/</code>). <code>G_FACTORY_INPUT</code> , <code>G_FACTORY_OUTPUT</code> e <code>G_FACTORY_DETERMINISTIC</code> são rótulos de invariante no <code>AGENTS.md</code> , não arquivos de portão.
6. SCRIPTS DETERMINÍSTICOS (0 LLM)	100% determinístico nas fases estruturais: análise de topologia, geração do Docker Compose com Traefik, particionamento de bancos lógicos PostgreSQL e geração de <code>.env</code> .
7. CAMPAINHAS DE ALERTA (Hooks)	Validações em tempo de execução via <code>contrato_factory.py</code> emitindo interrupção com traceback rastreável ao detectar ausência de dependências.

Dimensão	Detalhamento Técnico no Código
8. PESSOAS / PERSONAS (Agents)	Agente orquestrador de fábrica open-source e fatiador de integração VSA.
9. TAREFAS ÚNICAS (Skills)	Skills associadas: <code>aidd-factory-runner</code> , <code>aidd-open</code> , <code>open</code> , <code>fluxo-02-runner</code> .
10. TELEFONES EXTERNOS (MCPs)	Totalmente autônomo na orquestração de templates e fatias de código.
11. BILHETES (Rules / AGENTS.md)	<code>tools/aidd-factory/AGENTS.md</code> : Consumo exclusivo do envelope de infraestrutura canônico, Zero Stubs, reuso compulsório de <code>result.py</code> e <code>escritor_atomico.py</code> .

2. Evidência de Testes e Portões

- **Comando executado:** `pytest tools/aidd-factory/tests`
- **Resultado:** 19 testes passaram, 0 falhas em 94.95s.
- **Invariantes testadas:** Validação de planos de clínicas e delivery, compilação de compose e init_db, gateway generator, frontend factory real compilando sem erros e suporte a nicho dinâmico sem LLM.

3. Diagnóstico de Não-Conformidades

- Nenhum bug impeditivo. Módulo atende plenamente à governança e à Lei #11.

SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA

(Nenhum ticket impeditivo aberto. Módulo plenamente funcional).

SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)

- **Status do Módulo:** APROVADO & CERTIFICADO
- **Nível de Autonomia:** Pipeline de integração determinística multi-serviços com validação real de compilação.
- **Handoff:** Registrado no `manifesto_auditoria.json`. Próximo módulo: `aidd-bridge` (Fluxo 03).

Capítulo 5: AIDD Bridge — A Ponte da Libertação (Fluxo 03: Desacoplamento Low-Code)

Na Festa (A Metáfora Memorável)

O Chaveiro Libertador que resgata os brinquedos que estavam presos em gaiolas com cadeados caros de empresas distantes (Lovable, Supabase), limpando-os para funcionarem livres no quintal da sua própria casa.

Aqui a casa já existe — só que presa a um dono que cobra aluguel para você nem abrir a porta. Este é o chaveiro que destranca as paredes de vendor lock-in e devolve as chaves de verdade para o dono real do código.

Na Casa (A Foto Técnica Rigorosa e as 11 Dimensões)

- Caminho Físico no Disco: [tools/aidd-bridge](#)
- Arquivo Canônico de Regras: [tools/aidd-bridge/AGENTS.md](#)

Ficha de Auditoria Bit a Bit: [aidd-bridge](#) (Fluxo 03 — Low-Code / Desacoplamento)

Ferramenta: [tools/aidd-bridge](#)
Data da Auditoria: 2026-09-22
Status: AUDITADO / ÍNTEGRO
Responsável: Antigravity / Eco-AIDD Agent

SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)

1. Matriz Contratual das 11 Dimensões

Dimensão	Detalhamento Técnico no Código
1. RECEBE (Input)	CLI argparse: subcomandos scan , convert-db , merge , pack , migrate-auth , destroy , unpack . Consome projetos exportados do Lovable/v0/Bolt contendo React + Vite + Tailwind + migrações Supabase.
2. CRIA / PROCESSA	scanner.py (análise de páginas, rotas e migrações), data_bridge.py + sql_transpiler.py (sanitização de SQL Supabase para PostgreSQL padrão), unifier.py (agregação multi-app em fatias VSA), devops.py (geração de Dockerfile, Compose e Swarm), auth_migrator.py (migração de hashes de senha de auth.users), frontend_liberator.py (remoção de bibliotecas proprietárias), vsa_exporter.py (exportação do Quarteto Sine Qua Non dinâmico), pipeline_bridge.py (orquestração 6 fases).
3. ENTREGA (Output)	Aplicação 100% livre de lock-in, banco PostgreSQL consolidado (init-db.sql), stack Docker Swarm pronta para VPS, fatias verticais VSA em src/modules/ e diretório quarteto_sine_qua_non/ contendo Swagger, Webhook, MCP e Guia de Uso.
4. CONFIGURAÇÕES (Configs)	Flags CLI <code>--stack <lite\ full></code> , <code>--domain</code> , <code>--output</code> , <code>--traefik-network</code> , <code>--certresolver</code> , <code>--apply</code> , <code>--yes</code> , <code>--with-gottrue</code> .
5. GUARDAS (Gates)	Quality gates dedicados em gates/ : G_BRIDGE_VENDOR_LOCKIN , G_BRIDGE_DOCKER_OCI , G_BRIDGE_POSTGRESQL , G_BRIDGE_VSA_COMPAT .
6. SCRIPTS DETERMINÍSTICOS (0 LLM)	100% determinístico. Transpilação SQL por regex e gramática estática, manipulação de AST TypeScript/JavaScript para substituição de importações do cliente @supabase/supabase-js e empacotamento de templates Docker.
7. CAMPAINHAS DE ALERTA (Hooks)	_forcar_utf8_stdio para compatibilidade multi-plataforma (Windows cp1252), verificações de credenciais obrigatórias para ações remotas em VPS / Cloudflare DNS e prompt interativo obrigatório no comando destrutivo destroy .

Dimensão	Detalhamento Técnico no Código
8. PESSOAS / PERSONAS (Agents)	Agente especialista em desmonte de lock-in e unificação de apps legadas.
9. TAREFAS ÚNICAS (Skills)	Skills associadas: aidd-bridge-runner , aidd-freedom , freedom , fluxo-03-runner .
10. TELEFONES EXTERNOS (MCPs)	Módulo de automação de DNS com Cloudflare API (cloudflare_dns.py) e orquestração SSH para provisionamento de VPS.
11. BILHETES (Rules / AGENTS.md)	tools/aidd-bridge/AGENTS.md : Desacoplamento estrito, migração segura de credenciais, rollback em teardown e Quarteto Sine Qua Non mandatório.

2. Evidência de Testes e Portões

- **Comando executado:** `pytest tools/aidd-bridge/tests`
- **Resultado:** 64 testes passaram, 0 falhas em 1.19s.
- **Invariantes testadas:** Transpilação SQL Supabase → PostgreSQL, empacotamento de stack Swarm, unificação de fatias VSA, migração de hashes de auth e pipeline e2e do Fluxo 03.

3. Diagnóstico de Não-Conformidades

- Nenhum bug impeditivo. Módulo atende 100% às exigências do ecossistema.

SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA

(Nenhum ticket impeditivo aberto. Módulo plenamente funcional).

SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)

- **Status do Módulo:** APROVADO & CERTIFICADO
- **Nível de Autonomia:** Pipeline determinístico de 6 fases com extração limpa e zero dependência de IA para sanitização.
- **Handoff:** Registrado no [manifesto_auditoria.json](#). Próximo módulo: [aidd-master](#) (Meso-camada e convergência).

Capítulo 6: AIDD Master — O Maestro da Harmonização Monolítica VSA

Na Festa (A Metáfora Memorável)

O Maestro da Orquestra que garante que todos os instrumentos (fatias verticais) toquem em harmonia perfeita no mesmo salão, sem um bater no outro e com uma barreira invisível de segurança.

Com as construções de pé, é este maestro que garante que todas as alas do prédio funcionem como um único edifício coerente — nenhuma fatia vertical pisa no cano ou na fiação da vizinha.

Na Casa (A Foto Técnica Rigorosa e as 11 Dimensões)

- **Caminho Físico no Disco:** `tools/aidd-master`
- **Arquivo Canônico de Regras:** `tools/aidd-master/AGENTS.md`

Ficha de Auditoria Bit a Bit: [aidd-master](#) (Meso-Camada de Convergência Monolítica VSA)

Ferramenta: `tools/aidd-master`
Data da Auditoria: 2026-09-22
Status: AUDITADO / ÍNTEGRO
Responsável: Antigravity / Eco-AIDD Agent

SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)

1. Matriz Contratual das 11 Dimensões

Dimensão	Detalhamento Técnico no Código
1. RECEBE (Input)	Comandos CLI: <code>python scripts/aidd.py [add-module compose-orca run-all provision], dispatch_pipeline.py --planner <path.json> \ -dispatch <json></code> , manifestos de fatias VSA e contratos de handoff das três trilhas da Tríade.
2. CRIA / PROCESSA	Orquestração da Meso-Camada VSA: <code>dispatch_pipeline.py</code> (Kahn DAG para ordenação topológica de fatias), <code>engine_router.py</code> (despacho para o motor correto 01/02/03), <code>vsa_join_barrier.py</code> (barreira de validação síncrona com Git Worktrees efêmeras e merge na master), <code>orchestrator_pipeline.py</code> (orquestrador mestre), <code>add_module.py</code> (scaffolding de fatias VSA verticais independentes: router, service, repository, dtos, events).
3. ENTREGA (Output)	Monólito Modular VSA consolidado com SQLite WAL otimizado, barreira transacional unificada, contratos OpenAPI 3.1 tipados e convertidos para TypeScript (<code>openapi_to_ts.py</code>) e Quarteto Sine Qua Non dinâmico.
4. CONFIGURAÇÕES (Configs)	Configurações de pool SQLite WAL (<code>PRAGMA journal_mode=WAL</code>), limites de concorrência e barreira de tolerância a falhas (<code>--timeout</code> , <code>--retry</code> , <code>--fail-fast</code>).
5. GUARDAS (Gates)	12 quality gates determinísticos em <code>scripts/gates/</code> : <code>G_ARQUITETURA</code> , <code>G_AST_BOUNDED_CONTEXT</code> , <code>G_CHAOS</code> , <code>G_CONTRACTS</code> , <code>G ESTRUTURA</code> , <code>G_HARNESS_COMPAT</code> , <code>G_INJECT</code> , <code>G_PERFORMANCE</code> , <code>G_QUALIDADE</code> , <code>G_SEGREDOS</code> , <code>G_SEGURANCA</code> , <code>G_TESTES</code> .
6. SCRIPTS DETERMINÍSTICOS (0 LLM)	100% determinístico. Barreira de junção VSA baseada em Git Worktrees, ordenação topológica por algoritmo de Kahn, AST linter para isolamento de contextos delimitados (proibição de importações cruzadas não autorizadas) e Result Monad.
7. CAMPAINHAS DE ALERTA (Hooks)	Monitoramento de latência e p99 (<code>test_live.py</code> , <code>G_PERFORMANCE</code>), detecção de lock e retry automático em SQLite (<code>sqlite_busy_retry.py</code>), e bloqueio binário em caso de divergência de schema na Join Barrier.

Dimensão	Detalhamento Técnico no Código
8. PESSOAS / PERSONAS (Agents)	Agente orquestrador de convergência e maestro de fatias verticais VSA.
9. TAREFAS ÚNICAS (Skills)	Skills associadas: aidd-master-runner , aidd-dispatch-runner , aidd-pipeline-runner , aidd-orchestrator-runner .
10. TELEFONES EXTERNOS (MCPs)	Suporte completo a MCP Server SDK (test_mcp_server_sdk.py) permitindo expor as fatias do monólito como ferramentas MCP padronizadas.
11. BILHETES (Rules / AGENTS.md)	tools/aidd-master/AGENTS.md : Isolamento estrito de bounded contexts, Result Monad compulsório, SQLite WAL e proibição categórica de stubs.

2. Evidência de Testes e Portões

- **Comando executado:** `pytest tools/aidd-master/tests`
- **Resultado:** 439 testes passaram, 0 falhas, 3 pulados em 266.59s (04:26 min).
- **Invariantes testadas:** Bounded context AST isolation, VSA Join Barrier, outbox worker, SQLite busy retry, JWT hardening, Next.js exporter e validação dos 12 quality gates.

3. Diagnóstico de Não-Conformidades

- Nenhum bug impeditivo. Módulo é a coluna vertebral de convergência da Tríade e opera com máxima estabilidade.

SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA

(Nenhum ticket impeditivo aberto. Módulo plenamente funcional).

SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)

- **Status do Módulo:** APROVADO & CERTIFICADO
- **Nível de Autonomia:** Meso-camada determinística com barreira formal de sincronização e merge limpo de fatias VSA.
- **Handoff:** Registrado no [manifesto_auditoria.json](#). Próximo módulo: [aidd-enterprise](#).

Capítulo 7: AIDD Enterprise — O Cofre de Alta Segurança e Selos Criptográficos

Na Festa (A Metáfora Memorável)

O Inspetor do Cofre do Rei que confere o carimbo de ouro (assinatura digital SHA-256) em cada documento e garante que nenhum espião consiga alterar uma única linha de código.

É o inspetor que sela cada ambiente com um lacre inviolável (assinatura criptográfica) antes de qualquer chave ser entregue para o mundo real — se o lacre estiver quebrado, a entrega não sai.

Na Casa (A Foto Técnica Rigorosa e as 11 Dimensões)

- **Caminho Físico no Disco:** `tools/aidd-enterprise`

- **Arquivo Canônico de Regras:** [tools/aidd-enterprise/AGENTS.md](#)

Ficha de Auditoria Bit a Bit: [aidd-enterprise](#) (Resiliência Crítica SHA-256)

Ferramenta: [tools/aidd-enterprise](#)
Data da Auditoria: 2026-09-22
Status: AUDITADO / ÍNTEGRO
Responsável: Antigravity / Eco-AIDD Agent

SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)

1. Matriz Contratual das 11 Dimensões

Dimensão	Detalhamento Técnico no Código
1. RECEBE (Input)	Comandos CLI: <code>python scripts/aidd.py [add-module compose-orca run-all provision]</code> , componentes injetáveis (<code>scripts/injector/</code>) com validação via <code>component_manifest.json</code> e assinatura SHA-256.
2. CRIA / PROCESSA	Validação criptográfica de integridade de componentes, arquitetura Zero-Trust, isolamento de bounded contexts regulados, driver de eventos (<code>events_driver.py</code>), outbox worker transacional com idempotência, autenticação reforçada (JWT hardening, OIDC SSO, RLS).
3. ENTREGA (Output)	Monólito modular enterprise auditado, schemas OpenAPI 3.1 com tipagem TypeScript estrita, SQLite WAL com política de retenção e recovery, e trilhas de auditoria criptografadas.
4. CONFIGURAÇÕES (Configs)	Políticas de CSP (<code>security_csp_lib.py</code>), pool de conexões SQLite com retry determinístico (<code>sqlite_busy_retry.py</code>), limites de LRU de transações (<code>transaction_log_lru.py</code>).
5. GUARDAS (Gates)	10 quality gates locais em <code>scripts/gates/</code> : <code>G_ARQUITETURA</code> , <code>G_CHAOS</code> , <code>G_CONTRACTS</code> , <code>G ESTRUTURA</code> , <code>G_HARNESS_COMPAT</code> , <code>G_INJECT</code> , <code>G_PERFORMANCE</code> , <code>G_QUALIDADE</code> , <code>G_SEGREDOS</code> , <code>G_SEGURANCA</code> , <code>G_TESTES</code> .
6. SCRIPTS DETERMINÍSTICOS (0 LLM)	100% determinístico. Validação de hashes SHA-256, checagem AST de isolamento de bounded contexts, injeção de schemas sem dependência de inferência de IA e barreira Result Monad.
7. CAMPAINHAS DE ALERTA (Hooks)	<code>G_CHAOS</code> (injeção controlada de falhas para teste de resiliência), <code>G_SEGREDOS</code> (bloqueio de vazamento de credenciais) e <code>G_SEGURANCA</code> (defesa contra XSS/injeção SQL).
8. PESSOAS / PERSONAS (Agents)	Agente de conformidade enterprise e auditor de integridade criptográfica.
9. TAREFAS ÚNICAS (Skills)	Skills associadas: <code>aidd-enterprise-runner</code> , <code>aidd-diagnose</code> .

Dimensão	Detalhamento Técnico no Código
10. TELEFONES EXTERNOS (MCPs)	MCP Server SDK corporativo (test_mcp_server_sdk.py) e adaptadores de telemetria externa (OpenTelemetry , metrics.py).
11. BILHETES (Rules / AGENTS.md)	tools/aidd-enterprise/AGENTS.md : Integridade criptográfica compulsória, Result Monad em toda a camada de serviço, SQLite WAL e zero stubs.

2. Evidência de Testes e Portões

- **Comando executado:** `pytest tools/aidd-enterprise/tests`
- **Resultado:** 365 testes passaram, 0 falhas, 3 pulados em 148.87s (02:28 min).
- **Invariantes testadas:** Validação SHA-256 de manifestos, outbox idempotente, SSO OIDC, mitigação de XSS em webhooks, RLS e aprovação de todos os 10 quality gates.

3. Diagnóstico de Não-Conformidades

- Nenhum bug impeditivo. Módulo 100% íntegro e em estrita conformidade com as Leis do ecossistema.

SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA

(Nenhum ticket impeditivo aberto. Módulo plenamente funcional).

SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)

- **Status do Módulo:** APROVADO & CERTIFICADO
- **Nível de Autonomia:** Plataforma de missão crítica regulada 100% determinística.
- **Handoff:** Registrado no [manifesto_auditoria.json](#). Próximo módulo: [aidd-ops](#).

Capítulo 8: AIDD Ops — A Usina de Força e a Torre de Vigilância

Na Festa (A Metáfora Memorável)

O Chefe dos Engenheiros que constrói a usina de energia (servidor VPS), liga os motores (Docker), tranca as portas com chaves fortes (SSH seguro) e coloca câmeras de vigilância 24 horas por dia (Uptime Kuma).

É quem liga a energia do prédio já pronto, tranca as portas externas com fechaduras fortes e instala as câmeras de vigilância — para que a construção funcione 24 horas por dia sem ninguém arrombar a fechadura enquanto todos dormem.

Na Casa (A Foto Técnica Rigorosa e as 11 Dimensões)

- **Caminho Físico no Disco:** [tools/aidd-ops](#)
- **Arquivo Canônico de Regras:** [tools/aidd-ops/AGENTS.md](#)

Ficha de Auditoria Bit a Bit: [aidd-ops](#) (Infraestrutura, VPS e Observabilidade)

Ferramenta: [tools/aidd-ops](#)
Data da Auditoria: 2026-09-22
Status: AUDITADO / ÍNTEGRO
Responsável: Antigravity / Eco-AIDD Agent

SESSÃO I: ESTADO ATUAL (Diagnóstico Pré-Implementação)

1. Matriz Contratual das 11 Dimensões

Dimensão	Detalhamento Técnico no Código
1. RECEBE (Input)	CLI argparse: <code>python scripts/pipeline_ops.py [plan deploy monitor rotate-secrets]</code> . Parâmetros: <code>--texto</code> , <code>--nicho</code> , <code>--ferramentas-json</code> , <code>--dry-run</code> , <code>--vps-host</code> , <code>--vps-user</code> , <code>--ssh-key</code> .
2. CRIA / PROCESSA	Pipeline de provisionamento de infraestrutura em 3 fases de planejamento: Fase 1 (<code>01_intake.py</code> — detecção de nicho fixo ou dinâmico), Fase 2 (<code>02_curadoria.py</code> — seleção de stack open-source compatível), Fase 3 (<code>03_sizing.py</code> — dimensionamento de CPU/RAM/Disco por heurísticas determinísticas). Execução remota: <code>ssh_runner.py</code> (execução segura sem interpolação perigosa), <code>ansible</code> (hardening de SO e SSH via playbook idempotente), <code>uptime_kuma.py</code> (extração e exportação de probes reais de monitoramento), <code>rotate_secrets.py</code> (rotação de segredos via <code>sops</code> e <code>age</code>).
3. ENTREGA (Output)	<code>PLANO-INFRAESTRUTURA.json</code> em estrita conformidade com schema JSON, templates Docker Compose OCI, playbook Ansible executado, dashboard Uptime Kuma provisionado e credenciais criptografadas.
4. CONFIGURAÇÕES (Configs)	Parâmetros de CLI (<code>--dry-run</code> , <code>--ferramentas-json</code> , <code>-export-kuma</code>), catálogo de nichos (<code>data/catalogo_nichos.json</code>), requisitos de recursos (<code>data/requisitos_recursos.json</code>).
5. GUARDAS (Gates)	<code>G_INFRA_COMPOSE</code> (reprodutibilidade estrita dos manifests), <code>G_HADOLINT</code> (conformidade OCI dos Dockerfiles), <code>GateSshRunnerAST</code> (garantia de zero concatenação insegura em comandos remotos).
6. SCRIPTS DETERMINÍSTICOS (0 LLM)	100% determinístico. Dimensionamento de hardware por fórmulas lineares estáticas, parse e validação de compose via PyYAML, geração de playbooks e scripts de conexão SSH com Paramiko.
7. CAMPAINHAS DE ALERTA (Hooks)	Pré-voo SSH com Paramiko validando conectividade, autenticação e timeout antes de acionar playbooks; rejeição imediata se houver tentativa de injeção de parâmetros em comandos remotos.

Dimensão	Detalhamento Técnico no Código
8. PESSOAS / PERSONAS (Agents)	Agente orquestrador de operações de infraestrutura e observabilidade.
9. TAREFAS ÚNICAS (Skills)	Skills associadas: aidd-ops-runner , aidd-diagnose .
10. TELEFONES EXTERNOS (MCPs)	Módulos de conexão remota SSH (Paramiko/Ansible), Cloudflare DNS e integração com API do Uptime Kuma.
11. BILHETES (Rules / AGENTS.md)	tools/aidd-ops/AGENTS.md : Compose determinístico, observabilidade sem stubs, hardening via Ansible DevSec e suporte nativo ao envelope de nicho dinâmico.

2. Evidência de Testes e Portões

- **Comando executado:** `pytest tools/aidd-ops/tests`
- **Resultado:** 180 testes passaram, 0 falhas em 40.19s.
- **Invariantes testadas:** Hardening SSH/OS em dry-run, pré-voo Paramiko sem vazar credenciais, bloqueio AST de concatenação insegura, exportação e validação de monitores Uptime Kuma sem dados hardcoded.

3. Diagnóstico de Não-Conformidades

- Nenhum bug impeditivo. Módulo 100% íntegro e operacional.

SESSÃO II: PLANO DE CORREÇÃO & TICKETS DE MELHORIA

(Nenhum ticket impeditivo aberto. Módulo plenamente funcional).

SESSÃO III: ESTADO PÓS-IMPLEMENTAÇÃO (Certificação)

- **Status do Módulo:** APROVADO & CERTIFICADO
- **Nível de Autonomia:** Orquestrador determinístico de infraestrutura e observabilidade.
- **Handoff:** Registrado no [manifesto_auditoria.json](#). Todas as 8 ferramentas da pasta `tools/` auditadas com sucesso.

PARTE II: O CINTO DE UTILIDADES (AS 68 MICRO-FERRAMENTAS / SKILLS)

Na Festa

Imagine o cinto de utilidades do Batman ou a caixa de ferramentas mágica de um relojoeiro suíço. Cada uma dessas 68 ferramentas tem um formato único e resolve exatamente um problema específico sem fazer barulho nem desperdiçar energia.

Na Casa (O Catálogo Completo das 68 Skills Canônicas)

Localização canônica: [componentes/compartilhado/skills/](#)

1. Micro-Ferramenta: [agents-sdk](#)

- **Foto / Identidade:** [agents-sdk](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/agents-sdk/SKILL.md](#)
- **O que Faz (Missão Única):** Build, debug, or review Cloudflare Agents SDK applications using the agents package.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command ([/agents-sdk](#)) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py componentes sync --tipo todos](#).
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

2. Micro-Ferramenta: [aidd-auditor-4f-runner](#)

- **Foto / Identidade:** [aidd-auditor-4f-runner](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-auditor-4f-runner/SKILL.md](#)
- **O que Faz (Missão Única):** Motor agnóstico do Pipeline Linear de Auditoria 4 Fases (Inspetor, Arquiteto, Construtor, Retorno). Executa em Git Worktrees efêmeras.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command ([/aidd-auditor-4f-runner](#)) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).

5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

3. Micro-Ferramenta: `aidd-bridge`

- **Foto / Identidade:** `aidd-bridge`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-bridge/SKILL.md`
- **O que Faz (Missão Única):** Dispara o Fluxo 03 (Low-Code / Apps Unificadas | Slash: `/bridge`) da Tríade Canônica. Desmonte de lock-in Lovable/v0/Bolt e migração PostgreSQL.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-bridge`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

4. Micro-Ferramenta: `aidd-bridge-runner`

- **Foto / Identidade:** `aidd-bridge-runner`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-bridge-runner/SKILL.md`
- **O que Faz (Missão Única):** Extracts, unifies, and packages low-code projects (Lovable, v0, Bolt) for VPS deployment with PostgreSQL.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-bridge-runner`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.

11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

5. Micro-Ferramenta: [aidd-componentes](#)

- **Foto / Identidade:** [aidd-componentes](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-componentes/SKILL.md](#)
- **O que Faz (Missão Única):** Creates, updates, and synchronizes agnostic components across all ecosystem harnesses.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command ([/aidd-componentes](#)) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

6. Micro-Ferramenta: [aidd-dependencias](#)

- **Foto / Identidade:** [aidd-dependencias](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-dependencias/SKILL.md](#)
- **O que Faz (Missão Única):** Installs and verifies third-party skills and MCPs against [dependencias_externas.json](#).
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command ([/aidd-dependencias](#)) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

7. Micro-Ferramenta: [aidd-diagnose](#)

- **Foto / Identidade:** [aidd-diagnose](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-diagnose/SKILL.md](#)
- **O que Faz (Missão Única):** Systematic scientific fault triage using hypothesis isolation, regression testing, and code review graph.
- **As 11 Dimensões desta Micro-Ferramenta:**

1. Recebe (Input): Chamada de slash command (`/aidd-diagnose`) ou diretiva procedural no prompt.
2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

8. Micro-Ferramenta: `aidd-dispatch-runner`

- **Foto / Identidade:** `aidd-dispatch-runner`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-dispatch-runner/SKILL.md`
- **O que Faz (Missão Única):** Despacha fatias verticais VSA em Git Worktrees efêmeras com isolamento estrito, verificação de Quality Gates e convergência master.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-dispatch-runner`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

9. Micro-Ferramenta: `aidd-enterprise-runner`

- **Foto / Identidade:** `aidd-enterprise-runner`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-enterprise-runner/SKILL.md`
- **O que Faz (Missão Única):** Injects and audits mission-critical enterprise components with SHA-256 validation.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-enterprise-runner`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.

7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

10. Micro-Ferramenta: `aidd-evolucao-runner`

- **Foto / Identidade:** `aidd-evolucao-runner`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-evolucao-runner/SKILL.md`
- **O que Faz (Missão Única):** Motor agnóstico do Pipeline de Evolução Técnica gerado a partir do Plano de Evolução. Executa fases sequenciais de tickets em Git Worktrees efêmeras.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-evolucao-runner`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

11. Micro-Ferramenta: `aidd-factory-runner`

- **Foto / Identidade:** `aidd-factory-runner`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-factory-runner/SKILL.md`
- **O que Faz (Missão Única):** Application code and integration generator for multi-service stacks.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-factory-runner`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

12. Micro-Ferramenta: `aidd-forge-runner`

- **Foto / Identidade:** `aidd-forge-runner`

- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-forge-runner/SKILL.md](#)
- **O que Faz (Missão Única):** Executes bootstrap and governance hardening on target repositories using aidd-forge.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-forge-runner`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

13. Micro-Ferramenta: [aidd-freedom](#)

- **Foto / Identidade:** [aidd-freedom](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-freedom/SKILL.md](#)
- **O que Faz (Missão Única):** Dispara o Fluxo 03 (Low-Code / Apps Unificadas | Slash: `/freedom`) da Tríade Canônica. Desmonte de lock-in Lovable/v0/Bolt e migração PostgreSQL.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-freedom`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

14. Micro-Ferramenta: [aidd-generator-runner](#)

- **Foto / Identidade:** [aidd-generator-runner](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-generator-runner/SKILL.md](#)
- **O que Faz (Missão Única):** Triggers autonomous software generation via 8-phase pipeline using aidd-generator.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-generator-runner`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).

4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

15. Micro-Ferramenta: `aidd-grill`

- **Foto / Identidade:** `aidd-grill`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-grill/SKILL.md`
- **O que Faz (Missão Única):** Relentless Socratic interview protocol to resolve assumptions, trade-offs, and invariants before writing code.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-grill`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

16. Micro-Ferramenta: `aidd-grill-docs`

- **Foto / Identidade:** `aidd-grill-docs`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-grill-docs/SKILL.md`
- **O que Faz (Missão Única):** Socratic interview grounded in existing repository architecture, domain documentation, and invariant rules.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-grill-docs`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.

10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

17. Micro-Ferramenta: `aidd-handoff`

- **Foto / Identidade:** `aidd-handoff`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-handoff/SKILL.md`
- **O que Faz (Missão Única):** Serializes and compacts session state into a structured markdown artifact for context rotation or agent handover.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-handoff`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

18. Micro-Ferramenta: `aidd-livro-texto`

- **Foto / Identidade:** `aidd-livro-texto`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-livro-texto/SKILL.md`
- **O que Faz (Missão Única):** Generates and updates auditable corporate textbooks as PDF via pandoc and typst.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-livro-texto`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: `livro.py` em `scripts/`
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

19. Micro-Ferramenta: `aidd-master-runner`

- **Foto / Identidade:** `aidd-master-runner`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-master-runner/SKILL.md`
- **O que Faz (Missão Única):** Scaffolds and integrates clean vertical slices in aidd-master architecture.

- **As 11 Dimensões desta Micro-Ferramenta:**

1. Recebe (Input): Chamada de slash command (`/aidd-master-runner`) ou diretiva procedural no prompt.
2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

20. Micro-Ferramenta: `aidd-mcp`

- **Foto / Identidade:** `aidd-mcp`

- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-mcp/SKILL.md`

- **O que Faz (Missão Única):** Scaffolds and exposes new Model Context Protocol (MCP) servers across harnesses.

- **As 11 Dimensões desta Micro-Ferramenta:**

1. Recebe (Input): Chamada de slash command (`/aidd-mcp`) ou diretiva procedural no prompt.
2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

21. Micro-Ferramenta: `aidd-melhoria`

- **Foto / Identidade:** `aidd-melhoria`

- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-melhoria/SKILL.md`

- **O que Faz (Missão Única):** Deeply analyzes code improvements from natural language and produces structured evaluation reports.

- **As 11 Dimensões desta Micro-Ferramenta:**

1. Recebe (Input): Chamada de slash command (`/aidd-melhoria`) ou diretiva procedural no prompt.
2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.

6. Scripts 0 LLM: `analizador.py`, `cli.py`, `fallback.py`, `handoff.py`, `isolamento.py`, `observabilidade.py`, `rollback.py` em `scripts/`
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

22. Micro-Ferramenta: `aidd-open`

- **Foto / Identidade:** `aidd-open`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-open/SKILL.md`
- **O que Faz (Missão Única):** Dispara o Fluxo 02 (Motores Open-Source | Slash: `/open`) da Tríade Canônica. Curadoria e integração de engines open-source em fatias VSA.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-open`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

23. Micro-Ferramenta: `aidd-ops-runner`

- **Foto / Identidade:** `aidd-ops-runner`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-ops-runner/SKILL.md`
- **O que Faz (Missão Única):** Agentic infrastructure meta-orchestrator for cloud and container deployments.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-ops-runner`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

24. Micro-Ferramenta: `aidd-orca`

- **Foto / Identidade:** `aidd-orca`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-orca/SKILL.md`
- **O que Faz (Missão Única):** Executes multi-phase ORCA plans using isolated Git worktrees and quality gates.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-orca`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py componentes sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

25. Micro-Ferramenta: `aidd-orchestrate`

- **Foto / Identidade:** `aidd-orchestrate`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-orchestrate/SKILL.md`
- **O que Faz (Missão Única):** Routes plan execution between ORCA app, agent worktrees, and native execution engines.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-orchestrate`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py componentes sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

26. Micro-Ferramenta: `aidd-orchestrator-runner`

- **Foto / Identidade:** `aidd-orchestrator-runner`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-orchestrator-runner/SKILL.md`
- **O que Faz (Missão Única):** Orquestrador mestre síncrono da Tríade Canônica. Executa qualquer fluxo com validação formal de contratos de handoff.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-orchestrator-runner`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.

3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

27. Micro-Ferramenta: [aidd-pipeline-runner](#)

- **Foto / Identidade:** [aidd-pipeline-runner](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-pipeline-runner/SKILL.md](#)
- **O que Faz (Missão Única):** Executa pipelines determinísticos da Tríade em Git Worktrees efêmeras com barreira de sincronização (Join Barrier) a partir de planos Markdown ou manifestos JSON.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command ([/aidd-pipeline-runner](#)) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

28. Micro-Ferramenta: [aidd-plan](#)

- **Foto / Identidade:** [aidd-plan](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-plan/SKILL.md](#)
- **O que Faz (Missão Única):** Transforms analysis reports into structured audit/evolution plans in docs/planos/.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command ([/aidd-plan](#)) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.

9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

29. Micro-Ferramenta: [aidd-planner-runner](#)

- **Foto / Identidade:** [aidd-planner-runner](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-planner-runner/SKILL.md](#)
- **O que Faz (Missão Única):** Generates and validates canonical SDD/BDD project blueprints and fuels the Triad flows (Generator, Factory, Bridge).
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command ([/aidd-planner-runner](#)) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

30. Micro-Ferramenta: [aidd-planos](#)

- **Foto / Identidade:** [aidd-planos](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-planos/SKILL.md](#)
- **O que Faz (Missão Única):** Generates standard templates and drafts for audit, evolution, or test plans.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command ([/aidd-planos](#)) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

31. Micro-Ferramenta: [aidd-pure](#)

- **Foto / Identidade:** [aidd-pure](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-pure/SKILL.md](#)

- **O que Faz (Missão Única):** Dispara o Fluxo 01 (Do Zero Puro | Slash: /pure) da Tríade Canônica. Geração autoral via TDD Red-Green estrito e Monólito Modular VSA.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (/aidd-pure) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

32. Micro-Ferramenta: [aidd-skills](#)

- **Foto / Identidade:** [aidd-skills](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-skills/SKILL.md](#)
- **O que Faz (Missão Única):** Creates, optimizes, and evaluates agent skills across all ecosystem harnesses.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (/aidd-skills) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

33. Micro-Ferramenta: [aidd-spec](#)

- **Foto / Identidade:** [aidd-spec](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/aidd-spec/SKILL.md](#)
- **O que Faz (Missão Única):** Synthesizes discussions, requirements, and decisions into a deterministic, executable technical specification.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (/aidd-spec) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.

6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

34. Micro-Ferramenta: `aidd-tdd`

- **Foto / Identidade:** `aidd-tdd`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-tdd/SKILL.md`
- **O que Faz (Missão Única):** Strict Test-Driven Development protocol (Red-Green-Refactor) with zero stubs and polyglot runtime support.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-tdd`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

35. Micro-Ferramenta: `aidd-tickets`

- **Foto / Identidade:** `aidd-tickets`
- **Caminho no Disco:** `componentes/compartilhado/skills/aidd-tickets/SKILL.md`
- **O que Faz (Missão Única):** Decomposes specifications into atomic, incremental tracer-bullet tasks with bounded blast radius.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/aidd-tickets`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

36. Micro-Ferramenta: `cloudflare`

- **Foto / Identidade:** `cloudflare`
- **Caminho no Disco:** `componentes/compartilhado/skills/cloudflare/SKILL.md`
- **O que Faz (Missão Única):** Discover and choose Cloudflare products for apps, APIs, AI agents, storage, networking, and security. Use for architecture and product selection, including when the user describes a need without naming a Cloudflare product; then find the relevant skill or documentation.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/cloudflare`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

37. Micro-Ferramenta: `cloudflare-email-service`

- **Foto / Identidade:** `cloudflare-email-service`
- **Caminho no Disco:** `componentes/compartilhado/skills/cloudflare-email-service/SKILL.md`
- **O que Faz (Missão Única):** Implement or troubleshoot Cloudflare Email Sending and Email Routing integrations and their delivery configuration.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/cloudflare-email-service`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

38. Micro-Ferramenta: `cloudflare-one`

- **Foto / Identidade:** `cloudflare-one`
- **Caminho no Disco:** `componentes/compartilhado/skills/cloudflare-one/SKILL.md`
- **O que Faz (Missão Única):** Design, configure, troubleshoot, or review Cloudflare One Zero Trust and SASE deployments. Use `cloudflare-one-migrations` for migration planning from other vendors.

- **As 11 Dimensões desta Micro-Ferramenta:**

1. Recebe (Input): Chamada de slash command (`/cloudflare-one`) ou diretiva procedural no prompt.
2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

39. Micro-Ferramenta: `cloudflare-one-migrations`

- **Foto / Identidade:** `cloudflare-one-migrations`

- **Caminho no Disco:** `componentes/compartilhado/skills/cloudflare-one-migrations/SKILL.md`

- **O que Faz (Missão Única):** Assess and plan migrations from existing VPN, SWG, or SASE platforms to Cloudflare One, including policy mapping, parity gaps, and rollout.

- **As 11 Dimensões desta Micro-Ferramenta:**

1. Recebe (Input): Chamada de slash command (`/cloudflare-one-migrations`) ou diretiva procedural no prompt.
2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

40. Micro-Ferramenta: `componentes-runner`

- **Foto / Identidade:** `componentes-runner`

- **Caminho no Disco:** `componentes/compartilhado/skills/componentes-runner/SKILL.md`

- **O que Faz (Missão Única):** Creates, updates, and synchronizes agnostic components across all ecosystem harnesses.

- **As 11 Dimensões desta Micro-Ferramenta:**

1. Recebe (Input): Chamada de slash command (`/componentes-runner`) ou diretiva procedural no prompt.
2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.

6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

41. Micro-Ferramenta: [debug-issue](#)

- **Foto / Identidade:** [debug-issue](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/debug-issue/SKILL.md](#)
- **O que Faz (Missão Única):** Systematically debug issues using graph-powered code navigation
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command ([/debug-issue](#)) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

42. Micro-Ferramenta: [dependencia-runner](#)

- **Foto / Identidade:** [dependencia-runner](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/dependencia-runner/SKILL.md](#)
- **O que Faz (Missão Única):** Installs and verifies third-party skills and MCPs against [dependencias_externas.json](#).
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command ([/dependencia-runner](#)) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

43. Micro-Ferramenta: `durable-objects`

- **Foto / Identidade:** `durable-objects`
- **Caminho no Disco:** `componentes/compartilhado/skills/durable-objects/SKILL.md`
- **O que Faz (Missão Única):** Build, debug, or review Cloudflare Durable Objects code for persistent state and coordination.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/durable-objects`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

44. Micro-Ferramenta: `explore-codebase`

- **Foto / Identidade:** `explore-codebase`
- **Caminho no Disco:** `componentes/compartilhado/skills/explore-codebase/SKILL.md`
- **O que Faz (Missão Única):** Navigate and understand codebase structure using the knowledge graph
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/explore-codebase`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

45. Micro-Ferramenta: `fluxo-01-runner`

- **Foto / Identidade:** `fluxo-01-runner`
- **Caminho no Disco:** `componentes/compartilhado/skills/fluxo-01-runner/SKILL.md`
- **O que Faz (Missão Única):** Executa de ponta a ponta o Fluxo 01 (Do Zero Puro) da Tríade Canônica de forma estritamente síncrona.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/fluxo-01-runner`) ou diretiva procedural no prompt.

2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

46. Micro-Ferramenta: [fluxo-02-runner](#)

- **Foto / Identidade:** [fluxo-02-runner](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/fluxo-02-runner/SKILL.md](#)
- **O que Faz (Missão Única):** Executa de ponta a ponta o Fluxo 02 (Motores Open-Source) da Tríade Canônica de forma estritamente síncrona.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command ([/fluxo-02-runner](#)) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

47. Micro-Ferramenta: [fluxo-03-runner](#)

- **Foto / Identidade:** [fluxo-03-runner](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/fluxo-03-runner/SKILL.md](#)
- **O que Faz (Missão Única):** Executa de ponta a ponta o Fluxo 03 (Low-Code / Apps Unificadas) da Tríade Canônica de forma estritamente síncrona.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command ([/fluxo-03-runner](#)) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).

8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

48. Micro-Ferramenta: [freedom](#)

- **Foto / Identidade:** [freedom](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/freedom/SKILL.md](#)
- **O que Faz (Missão Única):** Dispara o Fluxo 03 (Low-Code / Apps Unificadas | Slash: /freedom) da Tríade Canônica. Desmonte de lock-in Lovable/v0/Bolt e migração PostgreSQL.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (/freedom) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

49. Micro-Ferramenta: [impeccable](#)

- **Foto / Identidade:** [impeccable](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/impeccable/SKILL.md](#)
- **O que Faz (Missão Única):** Use when the user wants to design, redesign, shape, critique, audit, polish, clarify, distill, harden, optimize, adapt, animate, colorize, extract, or otherwise improve a frontend interface. Covers websites, landing pages, dashboards, product UI, app shells, components, forms, settings, onboarding, and empty states. Handles UX review, visual hierarchy, information architecture, cognitive load, accessibility, performance, responsive behavior, theming, anti-patterns, typography, fonts, spacing, layout, alignment, color, motion, micro-interactions, UX copy, error states, edge cases, i18n, and reusable design systems or tokens. Also use for bland designs that need to become bolder or more delightful, loud designs that should become quieter, live browser iteration on UI elements, or ambitious visual effects that should feel technically extraordinary. Not for backend-only or non-UI tasks.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (/impeccable) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.

9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

50. Micro-Ferramenta: [mcp-creator-runner](#)

- **Foto / Identidade:** [mcp-creator-runner](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/mcp-creator-runner/SKILL.md](#)
- **O que Faz (Missão Única):** Scaffolds and exposes new Model Context Protocol (MCP) servers across harnesses.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command ([/mcp-creator-runner](#)) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

51. Micro-Ferramenta: [melhoria](#)

- **Foto / Identidade:** [melhoria](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/melhoria/SKILL.md](#)
- **O que Faz (Missão Única):** Deeply analyzes code improvements from natural language and produces structured evaluation reports.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command ([/melhoria](#)) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

52. Micro-Ferramenta: [nextjs-on-cloudflare](#)

- **Foto / Identidade:** [nextjs-on-cloudflare](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/nextjs-on-cloudflare/SKILL.md](#)
- **O que Faz (Missão Única):** Build, migrate, and deploy Next.js apps on Cloudflare Workers with vinext. Use when starting a Next.js project on Cloudflare, moving an existing app to Workers,

choosing between vinext and OpenNext, or setting up vinext for Workers. For setup, migration, or deployment, install vinext's upstream skills with `npx skills add cloudflare/vinext` if missing, then read and follow the applicable skill and docs.

- **As 11 Dimensões desta Micro-Ferramenta:**

1. Recebe (Input): Chamada de slash command (`/nextjs-on-cloudflare`) ou diretiva procedural no prompt.
2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
4. Configs: Parâmetros no YAML frontmatter (`name, description`).
5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

53. Micro-Ferramenta: `open`

- **Foto / Identidade:** `open`

- **Caminho no Disco:** `componentes/compartilhado/skills/open/SKILL.md`

- **O que Faz (Missão Única):** Dispara o Fluxo 02 (Motores Open-Source | Slash: `/open`) da Tríade Canônica. Curadoria de motores e fatias verticais VSA.

- **As 11 Dimensões desta Micro-Ferramenta:**

1. Recebe (Input): Chamada de slash command (`/open`) ou diretiva procedural no prompt.
2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
4. Configs: Parâmetros no YAML frontmatter (`name, description`).
5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

54. Micro-Ferramenta: `orca-plan-orchestrator`

- **Foto / Identidade:** `orca-plan-orchestrator`

- **Caminho no Disco:** `componentes/compartilhado/skills/orca-plan-orchestrator/SKILL.md`

- **O que Faz (Missão Única):** Executes multi-phase ORCA plans using isolated Git worktrees and quality gates.

- **As 11 Dimensões desta Micro-Ferramenta:**

1. Recebe (Input): Chamada de slash command (`/orca-plan-orchestrator`) ou diretiva procedural no prompt.
2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
4. Configs: Parâmetros no YAML frontmatter (`name, description`).

5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
6. Scripts 0 LLM: `agent_spawner.py`, `circuit_breaker.py`, `flight_plan.py`, `gate_auditor.py`, `hooks.py`, `orca_real_plan.py`, `orchestrator_engine.py`, `plan_io.py`, `plan_parser.py`, `state_engine.py`, `subagent_plan.py`, `worktree_engine.py` em `scripts/`
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

55. Micro-Ferramenta: `orchestrate`

- **Foto / Identidade:** `orchestrate`
- **Caminho no Disco:** `componentes/compartilhado/skills/orchestrate/SKILL.md`
- **O que Faz (Missão Única):** Routes plan execution between ORCA app, agent worktrees, and native execution engines.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/orchestrate`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

56. Micro-Ferramenta: `plan`

- **Foto / Identidade:** `plan`
- **Caminho no Disco:** `componentes/compartilhado/skills/plan/SKILL.md`
- **O que Faz (Missão Única):** Transforms analysis reports into structured audit/evolution plans in docs/planos/.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/plan`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.

10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

57. Micro-Ferramenta: `planos-auditoria-runner`

- **Foto / Identidade:** `planos-auditoria-runner`
- **Caminho no Disco:** `componentes/compartilhado/skills/planos-auditoria-runner/SKILL.md`
- **O que Faz (Missão Única):** Generates standard templates and drafts for audit, evolution, or test plans.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/planos-auditoria-runner`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

58. Micro-Ferramenta: `pure`

- **Foto / Identidade:** `pure`
- **Caminho no Disco:** `componentes/compartilhado/skills/pure/SKILL.md`
- **O que Faz (Missão Única):** Dispara o Fluxo 01 (Do Zero Puro | Slash: `/pure`) da Tríade Canônica. Geração autoral via TDD Red-Green estrito e Monólito Modular VSA.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/pure`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

59. Micro-Ferramenta: `refactor-safely`

- **Foto / Identidade:** `refactor-safely`
- **Caminho no Disco:** `componentes/compartilhado/skills/refactor-safely/SKILL.md`
- **O que Faz (Missão Única):** Plan and execute safe refactoring using dependency analysis
- **As 11 Dimensões desta Micro-Ferramenta:**

1. Recebe (Input): Chamada de slash command (`/refactor-safely`) ou diretiva procedural no prompt.
2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

60. Micro-Ferramenta: `review-changes`

- **Foto / Identidade:** `review-changes`
- **Caminho no Disco:** `componentes/compartilhado/skills/review-changes/SKILL.md`
- **O que Faz (Missão Única):** Perform a structured code review using change detection and impact
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/review-changes`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

61. Micro-Ferramenta: `sandbox-migrate-to-next`

- **Foto / Identidade:** `sandbox-migrate-to-next`
- **Caminho no Disco:** `componentes/compartilhado/skills/sandbox-migrate-to-next/SKILL.md`
- **O que Faz (Missão Única):** Migrate Cloudflare Sandbox apps from stable @cloudflare/sandbox to @cloudflare/sandbox@next (SDK 1.0 preview). Use sandbox-next for apps already on the preview.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/sandbox-migrate-to-next`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.

7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

62. Micro-Ferramenta: `sandbox-next`

- **Foto / Identidade:** `sandbox-next`
- **Caminho no Disco:** `componentes/compartilhado/skills/sandbox-next/SKILL.md`
- **O que Faz (Missão Única):** Build or maintain Cloudflare Sandbox apps on @cloudflare/sandbox@next (SDK 1.0 preview). Use `sandbox-migrate-to-next` when porting a stable app.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/sandbox-next`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

63. Micro-Ferramenta: `sandbox-stable`

- **Foto / Identidade:** `sandbox-stable`
- **Caminho no Disco:** `componentes/compartilhado/skills/sandbox-stable/SKILL.md`
- **O que Faz (Missão Única):** Build or maintain Cloudflare Sandbox apps on the stable @cloudflare/sandbox package. Use `sandbox-next` for preview apps and `sandbox-migrate-to-next` for stable-to-preview migrations.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/sandbox-stable`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

64. Micro-Ferramenta: `skill-creator-runner`

- **Foto / Identidade:** `skill-creator-runner`
- **Caminho no Disco:** `componentes/compartilhado/skills/skill-creator-runner/SKILL.md`
- **O que Faz (Missão Única):** Creates, optimizes, and evaluates agent skills across all ecosystem harnesses.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/skill-creator-runner`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

65. Micro-Ferramenta: `turnstile-spin`

- **Foto / Identidade:** `turnstile-spin`
- **Caminho no Disco:** `componentes/compartilhado/skills/turnstile-spin/SKILL.md`
- **O que Faz (Missão Única):** Set up, repair, or migrate to Cloudflare Turnstile bot verification in an existing frontend and backend, including server-side Siteverify.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/turnstile-spin`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter (`name`, `description`).
 5. Gates: `G_SKILL_ROT.py` confere se todo caminho, script e comando citado no `SKILL.md` existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em `SKILL.md`.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`. Distribuição aos harnesses via `python ecossistema.py components sync --tipo todos`.
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

66. Micro-Ferramenta: `web-perf`

- **Foto / Identidade:** `web-perf`
- **Caminho no Disco:** `componentes/compartilhado/skills/web-perf/SKILL.md`
- **O que Faz (Missão Única):** Audit, diagnose, or optimize website loading and interaction performance, Core Web Vitals, and Lighthouse performance scores.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command (`/web-perf`) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em `SKILL.md` orientando a IA ou script local.

3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
9. Skills: Integrada ao ecossistema multi-harness.
10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

67. Micro-Ferramenta: [workers-best-practices](#)

- **Foto / Identidade:** [workers-best-practices](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/workers-best-practices/SKILL.md](#)
- **O que Faz (Missão Única):** Cloudflare Workers best practices for production applications. Use when writing, reviewing, or configuring Workers.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command ([/workers-best-practices](#)) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.
 10. MCPs: Compatível com chamadas de ferramentas de contexto.
 11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

68. Micro-Ferramenta: [wrangler](#)

- **Foto / Identidade:** [wrangler](#)
- **Caminho no Disco:** [componentes/compartilhado/skills/wrangler/SKILL.md](#)
- **O que Faz (Missão Única):** Run or troubleshoot Wrangler CLI commands and configure Worker projects for local development, deployment, and Cloudflare resource management.
- **As 11 Dimensões desta Micro-Ferramenta:**
 1. Recebe (Input): Chamada de slash command ([/wrangler](#)) ou diretiva procedural no prompt.
 2. Cria / Processa: Roteiro determinístico contido em [SKILL.md](#) orientando a IA ou script local.
 3. Entrega (Output): Artefato estruturado (Markdown, JSON ou diff de código).
 4. Configs: Parâmetros no YAML frontmatter ([name](#), [description](#)).
 5. Gates: [G_SKILL_ROT.py](#) confere se todo caminho, script e comando citado no [SKILL.md](#) existe de verdade.
 6. Scripts 0 LLM: Nenhum script próprio: a skill é um roteiro em [SKILL.md](#).
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#). Distribuição aos harnesses via [python ecossistema.py components sync --tipo todos](#).
 8. Agents: Personas especializadas convocadas conforme o contexto da tarefa.
 9. Skills: Integrada ao ecossistema multi-harness.

10. MCPs: Compatível com chamadas de ferramentas de contexto.
11. Rules: Subordinada à Lei #4 (Economia Extrema de Tokens) e Lei #9.

PARTE III: OS 50 GUARDIÕES INCORRUPΤÍVEIS (QUALITY GATES)

Na Festa

Imagine 50 cães de guarda robóticos sentados na saída da fábrica. Cada um tem um sensor diferente. Um cheira se tem segredo vazando, outro mede a espessura da parede, outro confere se tem botão quebrado e outro morde o pneu para ver se está furado. Se um único guarda latir (der exit 1), o portão de saída se tranca imediatamente e ninguém sai até consertar!

Na Casa (A Matriz dos 50 Quality Gates Canônicos)

Localização canônica: `gates/G_*.py`

1. Guarda Incorruptível: `G_amelhoria.py`

- **Foto / Identidade:** `G_amelhoria.py`
- **Caminho no Disco:** `gates/G_amelhoria.py`
- **Missão de Segurança:** Quality Gate Determinístico de Rótulo Honesto para aidd-melhoria.
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Argumentos CLI: `--relatorio`
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Não está no `.pre-commit-config.yaml`: roda sob demanda (`python gates/G_amelhoria.py`).
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

2. Guarda Incorruptível: `G_ANT_LOCKIN_LEGADO.py`

- **Foto / Identidade:** `G_ANT_LOCKIN_LEGADO.py`
- **Caminho no Disco:** `gates/G_ANT_LOCKIN_LEGADO.py`
- **Missão de Segurança:** Varre entrega/legado por resíduo lovable|supabase|firebase e dirs `.lovable/`.
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).

5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
6. Scripts 0 LLM: 100% mecânico e matemático.
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
8. Agents: Executa sem intervenção de agente cognitivo.
9. Skills: Invocado pelas skills de runner e auditoria.
10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

3. Guarda Incorrūptível: `G_ARQUITETURA_DELIVERABLE.py`

- **Foto / Identidade:** `G_ARQUITETURA_DELIVERABLE.py`
- **Caminho no Disco:** `gates/G_ARQUITETURA_DELIVERABLE.py`
- **Missão de Segurança:** Gate estatico via AST que valida conformidade com Clean Architecture e DDD
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

4. Guarda Incorrūptível: `G_CLI_HELP_CONSISTENCIA.py`

- **Foto / Identidade:** `G_CLI_HELP_CONSISTENCIA.py`
- **Caminho no Disco:** `gates/G_CLI_HELP_CONSISTENCIA.py`
- **Missão de Segurança:** Detecta divergência entre flags de CLI realmente definidas via
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

5. Guarda Incorrūptível: `G_COMPONENTE_AGNOSTICO.py`

- **Foto / Identidade:** `G_COMPONENTE_AGNOSTICO.py`
- **Caminho no Disco:** `gates/G_COMPONENTE_AGNOSTICO.py`

- **Missão de Segurança:** Audita se todo componente novo ou modificado (detectado via git diff / status)
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Argumentos CLI: `--base, --todos`
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

6. Guarda Incorrupível: `G_CONTRACT_ROT.py`

- **Foto / Identidade:** `G_CONTRACT_ROT.py`
- **Caminho no Disco:** `gates/G_CONTRACT_ROT.py`
- **Missão de Segurança:** Portão determinístico de prevenção de Contract Rot.
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Argumentos CLI: `--committed-spec, --projeto, --url`
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

7. Guarda Incorrupível: `G_DEPENDENCIAS_PIN_HASH.py`

- **Foto / Identidade:** `G_DEPENDENCIAS_PIN_HASH.py`
- **Caminho no Disco:** `gates/G_DEPENDENCIAS_PIN_HASH.py`
- **Missão de Segurança:** PLAN-0018 fase 04 (pin-exato-e-hashes-requirements-lockfile — SEC-4):
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.

7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
8. Agents: Executa sem intervenção de agente cognitivo.
9. Skills: Invocado pelas skills de runner e auditoria.
10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

8. Guarda Incorruptível: `G_DETERMINISMO_LEI_1.py`

- **Foto / Identidade:** `G_DETERMINISMO_LEI_1.py`
- **Caminho no Disco:** `gates/G_DETERMINISMO_LEI_1.py`
- **Missão de Segurança:** Auditoria de Determinismo Mecânico: Bloqueia o uso de SDKs e chamadas de LLM
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

9. Guarda Incorruptível: `G_DISCIPLINA_TESTE_FERRAMENTA.py`

- **Foto / Identidade:** `G_DISCIPLINA_TESTE_FERRAMENTA.py`
- **Caminho no Disco:** `gates/G_DISCIPLINA_TESTE_FERRAMENTA.py`
- **Missão de Segurança:** Auditoria de Disciplina de Teste e Validação de Ferramentas.
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Argumentos CLI: `--files`
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

10. Guarda Incorruptível: `G_DISPATCH_PIPELINE_VSA.py`

- **Foto / Identidade:** `G_DISPATCH_PIPELINE_VSA.py`
- **Caminho no Disco:** `gates/G_DISPATCH_PIPELINE_VSA.py`
- **Missão de Segurança:** Portão determinístico de validação formal de manifestos de despacho topológico

- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Argumentos CLI: `--manifesto`
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Não está no `.pre-commit-config.yaml`: roda sob demanda (`python gates/G_DISPATCH_PIPELINE_VSA.py`).
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

11. Guarda Incorrúptível: `G_DOCS_ROT.py`

- **Foto / Identidade:** `G_DOCS_ROT.py`
- **Caminho no Disco:** `gates/G_DOCS_ROT.py`
- **Missão de Segurança:** Verifica:
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Não está no `.pre-commit-config.yaml`: roda sob demanda (`python gates/G_DOCS_ROT.py`).
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

12. Guarda Incorrúptível: `G_DRIFT_ANALYZER.py`

- **Foto / Identidade:** `G_DRIFT_ANALYZER.py`
- **Caminho no Disco:** `gates/G_DRIFT_ANALYZER.py`
- **Missão de Segurança:** Detecta redundâncias estruturais e duplicidade de assinaturas de funções
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Argumentos CLI: `--strict`, `--target`
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.

8. Agents: Executa sem intervenção de agente cognitivo.
9. Skills: Invocado pelas skills de runner e auditoria.
10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

13. Guarda Incorrúptível: [G_DRIFT_NUCLEO_COMPARTILHADO.py](#)

- **Foto / Identidade:** [G_DRIFT_NUCLEO_COMPARTILHADO.py](#)
- **Caminho no Disco:** [gates/G_DRIFT_NUCLEO_COMPARTILHADO.py](#)
- **Missão de Segurança:** Detecta divergência silenciosa entre pares de diretórios que nasceram da
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate [G_PORTAO_PROVA_QUE_MORDE.py](#).
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#).
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

14. Guarda Incorrúptível: [G_ECOSSISTEMA_INTEGRIDADE.py](#)

- **Foto / Identidade:** [G_ECOSSISTEMA_INTEGRIDADE.py](#)
- **Caminho no Disco:** [gates/G_ECOSSISTEMA_INTEGRIDADE.py](#)
- **Missão de Segurança:** Validação determinística de integridade do meta-repositório ecossistema-aidd.
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate [G_PORTAO_PROVA_QUE_MORDE.py](#).
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#).
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

15. Guarda Incorrúptível: [G_ENV_ROT.py](#)

- **Foto / Identidade:** [G_ENV_ROT.py](#)
- **Caminho no Disco:** [gates/G_ENV_ROT.py](#)
- **Missão de Segurança:** Quality Gate determinístico de prevenção a Config & Environment Rot.
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.

- **As 11 Dimensões deste Quality Gate:**

1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
4. Configs: Argumentos CLI: `--env-file`, `--root-dir`
5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
6. Scripts 0 LLM: 100% mecânico e matemático.
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
8. Agents: Executa sem intervenção de agente cognitivo.
9. Skills: Invocado pelas skills de runner e auditoria.
10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

16. Guarda Incorrupível: `G_ESCRITOR_ATOMICO.py`

- **Foto / Identidade:** `G_ESCRITOR_ATOMICO.py`
- **Caminho no Disco:** `gates/G_ESCRITOR_ATOMICO.py`
- **Missão de Segurança:** Verificação determinística (AST) de que os pontos críticos de gravação de
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Não está no `.pre-commit-config.yaml`: roda sob demanda (`python gates/G_ESCRITOR_ATOMICO.py`).
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

17. Guarda Incorrupível: `G ESTRUTURA_ESTADO.py`

- **Foto / Identidade:** `G ESTRUTURA_ESTADO.py`
- **Caminho no Disco:** `gates/G ESTRUTURA_ESTADO.py`
- **Missão de Segurança:** Auditoria de Persistência Estruturada de Estado de Orquestração.
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Argumentos CLI: `--check-artifact`, `--type`
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.

10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

18. Guarda Incorruptível: `G_FRONTEND_LAYERS.py`

- **Foto / Identidade:** `G_FRONTEND_LAYERS.py`
- **Caminho no Disco:** `gates/G_FRONTEND_LAYERS.py`
- **Missão de Segurança:** Validação estática de separação de camadas nas entregas e templates de Frontend.
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

19. Guarda Incorruptível: `G_HADOLINT.py`

- **Foto / Identidade:** `G_HADOLINT.py`
- **Caminho no Disco:** `gates/G_HADOLINT.py`
- **Missão de Segurança:** Audita estaticamente todos os Dockerfiles do repositório (e os gerados pelas
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

20. Guarda Incorruptível: `G_HANDOFF_MELHORIA.py`

- **Foto / Identidade:** `G_HANDOFF_MELHORIA.py`
- **Caminho no Disco:** `gates/G_HANDOFF_MELHORIA.py`
- **Missão de Segurança:** Portão de transição melhoria → plan consumido pelo orquestrador.
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**

1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
4. Configs: Argumentos CLI: `--handoff`, `--repo-root`
5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
6. Scripts 0 LLM: 100% mecânico e matemático.
7. Hooks: Não está no `.pre-commit-config.yaml`: roda sob demanda (`python gates/G_HANDOFF_MELHORIA.py`).
8. Agents: Executa sem intervenção de agente cognitivo.
9. Skills: Invocado pelas skills de runner e auditoria.
10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

21. Guarda Incorrupível: `G_HARNESS_COMPAT.py`

- **Foto / Identidade:** `G_HARNESS_COMPAT.py`
- **Caminho no Disco:** `gates/G_HARNESS_COMPAT.py`
- **Missão de Segurança:** Materializa o gate `G_HARNESS_COMPAT` que o plano de execução original
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

22. Guarda Incorrupível: `G_HONESTIDADE_ROTULO.py`

- **Foto / Identidade:** `G_HONESTIDADE_ROTULO.py`
- **Caminho no Disco:** `gates/G_HONESTIDADE_ROTULO.py`
- **Missão de Segurança:** Verificacao mecanica da Regra de Ouro #9 (AGENTS.md §2, "Honestidade de
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.

10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

23. Guarda Incorruptível: `G_IDIOMA_LEI_4.py`

- **Foto / Identidade:** `G_IDIOMA_LEI_4.py`
- **Caminho no Disco:** `gates/G_IDIOMA_LEI_4.py`
- **Missão de Segurança:** Verificação mecânica determinística da Lei #4 (AGENTS.md §2, "Extreme Token
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Argumentos CLI: `--caminho`
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

24. Guarda Incorruptível: `G_INFRA_COMPOSE.py`

- **Foto / Identidade:** `G_INFRA_COMPOSE.py`
- **Caminho no Disco:** `gates/G_INFRA_COMPOSE.py`
- **Missão de Segurança:** Audita a integridade estática, sintática, de segurança e topológica dos arquivos
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

25. Guarda Incorruptível: `G_ISOLATION_AUDIT.py`

- **Foto / Identidade:** `G_ISOLATION_AUDIT.py`
- **Caminho no Disco:** `gates/G_ISOLATION_AUDIT.py`
- **Missão de Segurança:** Auditoria estática de isolamento de fatias verticais (Vertical Slice Architecture - VSA).
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.

- **As 11 Dimensões deste Quality Gate:**

1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
6. Scripts 0 LLM: 100% mecânico e matemático.
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
8. Agents: Executa sem intervenção de agente cognitivo.
9. Skills: Invocado pelas skills de runner e auditoria.
10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

26. Guarda Incorrupível: `G_LAYOUT_ENTREGA.py`

- **Foto / Identidade:** `G_LAYOUT_ENTREGA.py`
- **Caminho no Disco:** `gates/G_LAYOUT_ENTREGA.py`
- **Missão de Segurança:** Bloqueia entrega do usuário gerada sob `<clone>/projetos/` quando existe
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

27. Guarda Incorrupível: `G_LEI_DECLARA_PORTAO.py`

- **Foto / Identidade:** `G_LEI_DECLARA_PORTAO.py`
- **Caminho no Disco:** `gates/G_LEI_DECLARA_PORTAO.py`
- **Missão de Segurança:** Meta-Quality Gate que audita e bloqueia se qualquer Lei Inviolável em AGENTS.md
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Argumentos CLI: `--agents-file`
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.

11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

28. Guarda Incorrúptível: `G_LIVRO_EVIDENCIA.py`

- **Foto / Identidade:** `G_LIVRO_EVIDENCIA.py`
- **Caminho no Disco:** `gates/G_LIVRO_EVIDENCIA.py`
- **Missão de Segurança:** Audita deterministicamente o livro-texto gerado para um projeto, impedindo que
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Argumentos CLI: `--json`, `--livro`, `--projeto`
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Registrado no `.pre-commit-config.yaml` com `stages: [manual]` (roda só quando chamado).
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

29. Guarda Incorrúptível: `G_LLM_PROMPT_SHIELD.py`

- **Foto / Identidade:** `G_LLM_PROMPT_SHIELD.py`
- **Caminho no Disco:** `gates/G_LLM_PROMPT_SHIELD.py`
- **Missão de Segurança:** Auditoria de Blindagem e Sanitização Anti-Prompt Injection em Clientes LLM.
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

30. Guarda Incorrúptível: `G_MIGRATION_ROT.py`

- **Foto / Identidade:** `G_MIGRATION_ROT.py`
- **Caminho no Disco:** `gates/G_MIGRATION_ROT.py`
- **Missão de Segurança:** Quality Gate determinístico de prevenção a Migration Rot.
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**

1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
4. Configs: Argumentos CLI: `--target`
5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
6. Scripts 0 LLM: 100% mecânico e matemático.
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
8. Agents: Executa sem intervenção de agente cognitivo.
9. Skills: Invocado pelas skills de runner e auditoria.
10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

31. Guarda IncorrupΤível: `G_ORQUESTADOR_SINCRONO.py`

- **Foto / Identidade:** `G_ORQUESTADOR_SINCRONO.py`
- **Caminho no Disco:** `gates/G_ORQUESTADOR_SINCRONO.py`
- **Missão de Segurança:** Quality Gate: `G_ORQUESTADOR_SINCRONO`
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Não está no `.pre-commit-config.yaml`: roda sob demanda (`python gates/G_ORQUESTADOR_SINCRONO.py`).
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

32. Guarda IncorrupΤível: `G_PACOTE_CORE.py`

- **Foto / Identidade:** `G_PACOTE_CORE.py`
- **Caminho no Disco:** `gates/G_PACOTE_CORE.py`
- **Missão de Segurança:** Falha se a árvore de distribuição (export / pacote usuário) contiver artefatos
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.

11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

33. Guarda Incorrutível: `G_PIPELINE_HANDOFF.py`

- **Foto / Identidade:** `G_PIPELINE_HANDOFF.py`
- **Caminho no Disco:** `gates/G_PIPELINE_HANDOFF.py`
- **Missão de Segurança:** Portão determinístico de validação formal de manifestos de handoff de execução
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Argumentos CLI: `--manifesto`
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

34. Guarda Incorrutível: `G_PORTAO_PROVA_QUE_MORDE.py`

- **Foto / Identidade:** `G_PORTAO_PROVA_QUE_MORDE.py`
- **Caminho no Disco:** `gates/G_PORTAO_PROVA_QUE_MORDE.py`
- **Missão de Segurança:** Meta-Quality Gate que audita e bloqueia qualquer gate que seja entregue
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

35. Guarda Incorrutível: `G_PROTOCOL_FALLBACK.py`

- **Foto / Identidade:** `G_PROTOCOL_FALLBACK.py`
- **Caminho no Disco:** `gates/G_PROTOCOL_FALLBACK.py`
- **Missão de Segurança:** Auditoria de Paridade e Fallback REST vs Model Context Protocol (MCP).
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).

3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
6. Scripts 0 LLM: 100% mecânico e matemático.
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
8. Agents: Executa sem intervenção de agente cognitivo.
9. Skills: Invocado pelas skills de runner e auditoria.
10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

36. Guarda Incorrupível: `G_PROTOTYPE_REWRITE.py`

- **Foto / Identidade:** `G_PROTOTYPE_REWRITE.py`
- **Caminho no Disco:** `gates/G_PROTOTYPE_REWRITE.py`
- **Missão de Segurança:** Garante o isolamento determinístico de protótipos em sandbox/ e impede a
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Argumentos CLI: `--target`
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

37. Guarda Incorrupível: `G_QUARTETO_SINE_QUA_NON.py`

- **Foto / Identidade:** `G_QUARTETO_SINE_QUA_NON.py`
- **Caminho no Disco:** `gates/G_QUARTETO_SINE_QUA_NON.py`
- **Missão de Segurança:** Portão determinístico de auditoria do Quarteto Sine Qua Non Dinâmico.
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Argumentos CLI: `--target`
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

38. Guarda Incorrupível: [G_RESUMO_USUARIO.py](#)

- **Foto / Identidade:** [G_RESUMO_USUARIO.py](#)
- **Caminho no Disco:** [gates/G_RESUMO_USUARIO.py](#)
- **Missão de Segurança:** Exige, quando existe raiz de entrega com README-USUARIO.md, também
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta [exit 1](#).
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git ([--files](#)).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso ([exit 0](#) = APROVADO, [exit 1](#) = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate [G_PORTAO_PROVA_QUE_MORDE.py](#).
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#).
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

39. Guarda Incorrupível: [G_SAIDA_BINARIA.py](#)

- **Foto / Identidade:** [G_SAIDA_BINARIA.py](#)
- **Caminho no Disco:** [gates/G_SAIDA_BINARIA.py](#)
- **Missão de Segurança:** Auditoria de Qualidade Binária: Verifica deterministicamente via AST que todos
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta [exit 1](#).
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git ([--files](#)).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso ([exit 0](#) = APROVADO, [exit 1](#) = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate [G_PORTAO_PROVA_QUE_MORDE.py](#).
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via [.pre-commit-config.yaml](#).
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

40. Guarda Incorrupível: [G_SEGREDOS.py](#)

- **Foto / Identidade:** [G_SEGREDOS.py](#)
- **Caminho no Disco:** [gates/G_SEGREDOS.py](#)
- **Missão de Segurança:** Escaneia TODOS os arquivos rastreados pelo git (na raiz do ecossistema, não
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta [exit 1](#).
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git ([--files](#)).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).

3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
6. Scripts 0 LLM: 100% mecânico e matemático.
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
8. Agents: Executa sem intervenção de agente cognitivo.
9. Skills: Invocado pelas skills de runner e auditoria.
10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

41. Guarda Incorrūptível: `G_SKILL_ROT.py`

- **Foto / Identidade:** `G_SKILL_ROT.py`
- **Caminho no Disco:** `gates/G_SKILL_ROT.py`
- **Missão de Segurança:** Quality Gate determinístico de prevenção a Skill & Tool Rot.
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Argumentos CLI: `--repo-root`, `--skills-dir`
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

42. Guarda Incorrūptível: `G_STACK_PADRAO_OURO.py`

- **Foto / Identidade:** `G_STACK_PADRAO_OURO.py`
- **Caminho no Disco:** `gates/G_STACK_PADRAO_OURO.py`
- **Missão de Segurança:** Portão determinístico do Padrão-Ouro de Stack Tecnológica (Lei Canônica #11).
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Argumentos CLI: `--target`
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

43. Guarda IncorrupΤίvel: `g_SUPPLY_CHAIN.py`

- **Foto / Identidade:** `g_SUPPLY_CHAIN.py`
- **Caminho no Disco:** `gates/G_SUPPLY_CHAIN.py`
- **Missão de Segurança:** PLAN-0031 fase 01: Verificação da integridade da cadeia de suprimentos e
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Não está no `.pre-commit-config.yaml`: roda sob demanda (`python gates/G_SUPPLY_CHAIN.py`).
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

44. Guarda IncorrupΤίvel: `g_SYNC_CMD_ROT.py`

- **Foto / Identidade:** `g_SYNC_CMD_ROT.py`
- **Caminho no Disco:** `gates/G_SYNC_CMD_ROT.py`
- **Missão de Segurança:** Bloqueia a forma incompleta de sincronizacao de componentes em docs vivos
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

45. Guarda IncorrupΤίvel: `g_TEMPLATE_FORGE_ROT.py`

- **Foto / Identidade:** `g_TEMPLATE_FORGE_ROT.py`
- **Caminho no Disco:** `gates/G_TEMPLATE_FORGE_ROT.py`
- **Missão de Segurança:** Portão determinístico de integridade dos templates do aidd-forge.
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).

3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
6. Scripts 0 LLM: 100% mecânico e matemático.
7. Hooks: Não está no `.pre-commit-config.yaml`: roda sob demanda (`python gates/G_TEMPLATE_FORGE_ROT.py`).
8. Agents: Executa sem intervenção de agente cognitivo.
9. Skills: Invocado pelas skills de runner e auditoria.
10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

46. Guarda Incorrupível: `G_TESTES_REAIS.py`

- **Foto / Identidade:** `G_TESTES_REAIS.py`
- **Caminho no Disco:** `gates/G_TESTES_REAIS.py`
- **Missão de Segurança:** Roda pytest de verdade em cada tools/ e falha (`exit 1`) se
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

47. Guarda Incorrupível: `G_TRANSACTION_LOG_LRU.py`

- **Foto / Identidade:** `G_TRANSACTION_LOG_LRU.py`
- **Caminho no Disco:** `gates/G_TRANSACTION_LOG_LRU.py`
- **Missão de Segurança:** Verificação determinística (AST + hashes SHA-256) da entrega do item
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Não está no `.pre-commit-config.yaml`: roda sob demanda (`python gates/G_TRANSACTION_LOG_LRU.py`).
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

48. Guarda Incorrupível: `g_UNIVERSAL_HARNESS.py`

- **Foto / Identidade:** `g_UNIVERSAL_HARNESS.py`
- **Caminho no Disco:** `gates/g_UNIVERSAL_HARNESS.py`
- **Missão de Segurança:** Audita a garantia SINE QUA NON de que todo componente, skill, MCP e hook
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `g_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

49. Guarda Incorrupível: `g_USER_FACING_PTBR.py`

- **Foto / Identidade:** `g_USER_FACING_PTBR.py`
- **Caminho no Disco:** `gates/g_USER_FACING_PTBR.py`
- **Missão de Segurança:** Bloqueia jargão não explicado em superfícies lidas pelo usuário leigo
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).
 4. Configs: Sem argumentos de CLI (roda sobre o repositório inteiro).
 5. Gates: Autovalidado pelo meta-gate `g_PORTAO_PROVA_QUE_MORDE.py`.
 6. Scripts 0 LLM: 100% mecânico e matemático.
 7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
 8. Agents: Executa sem intervenção de agente cognitivo.
 9. Skills: Invocado pelas skills de runner e auditoria.
 10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
 11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

50. Guarda Incorrupível: `g_ZERO_HEADLESS.py`

- **Foto / Identidade:** `g_ZERO_HEADLESS.py`
- **Caminho no Disco:** `gates/g_ZERO_HEADLESS.py`
- **Missão de Segurança:** Gate de Enforcement Real contra Subagentes Headless e Concorrência Paralela
- **Prova que Morde (Lei #13):** Testado por teste automatizado dedicado que provoca a violação sintética e asserta `exit 1`.
- **As 11 Dimensões deste Quality Gate:**
 1. Recebe (Input): Árvore de arquivos do repositório ou changeset git (`--files`).
 2. Cria / Processa: Análise estática por AST Python, regex ou schema JSON (Zero LLM).
 3. Entrega (Output): Código binário rigoroso (`exit 0` = APROVADO, `exit 1` = BLOQUEADO).

4. Configs: Argumentos CLI: `--dangerously-force-headless`
5. Gates: Autovalidado pelo meta-gate `G_PORTAO_PROVA_QUE_MORDE.py`.
6. Scripts 0 LLM: 100% mecânico e matemático.
7. Hooks: Roda automaticamente no commit via `.pre-commit-config.yaml`.
8. Agents: Executa sem intervenção de agente cognitivo.
9. Skills: Invocado pelas skills de runner e auditoria.
10. MCPs: Nenhum — o portão não depende de servidor MCP para rodar.
11. Rules: Subordinado à Lei #2 (Saída Estritamente Binária).

PARTE IV: OS 141 COMPONENTES AGNÓSTICOS (PEÇAS UNIVERSAIS LEGO)

Na Festa

Imagine peças de LEGO de altíssima precisão. Não importa se você está montando na mesa azul (Claude), na mesa verde (Gemini), na mesa cinza (Cursor) ou na mesa preta (OpenCode). As peças encaixam com o mesmo clique suave e a mesma firmeza matemática. Nenhuma mesa fica com uma peça diferente da outra.

Na Casa (A Sincronização dos 141 Componentes)

- **Fonte Canônica Única:** `componentes/compartilhado/`
- **Ambientes Espelhados (Harnesses):** `.agents/`, `.claude/`, `.gemini/`, `.cursor/`, `.opencode/`, `.mimocode/`, `.codebuddy/`.
- **Garantia de Integridade:** Validação criptográfica SHA-256 via `python ecossistema.py components verify --tipo todos`. Se um único byte divergir entre qualquer harness e a fonte canônica, o sistema aponta drift e bloqueia o commit.

As 6 Famílias dos 141 Componentes:

1. **Slash Commands Canônicos** (`componentes/compartilhado/comandos/`): 16 especificações de comando em Markdown (`/pure`, `/forge`, `/melhoria`, etc.).
2. **Campainhas e Travas Git** (`componentes/compartilhado/hooks/`): 9 itens — travas de pré-commit e verificação de regras.
3. **Escudo de Segurança** (`componentes/compartilhado/security/`): 2 itens — detectores de vazamento de segredos e chaves.
4. **Habilidades Atômicas** (`componentes/compartilhado/skills/`): 68 pastas de skills padronizadas.
5. **Esquemas e Contratos Formais** (`componentes/compartilhado/specs/`): 8 esquemas JSON Draft-7 para handoff formal entre módulos.
6. **Núcleo Compartilhado de Código** (`componentes/compartilhado/src-core/`): 38 itens — utilitários de missão crítica (`escritor_atomico.py`, `result.py`, `assinatura_manifesto.py`).

Errata das Fichas de 2026-09-22

As fichas da Parte I são o registro histórico da auditoria de 2026-09-22. Estes nomes estavam errados nelas e foram corrigidos ao gerar este livro (o arquivo histórico não foi alterado):

- **aidd-forge:** `G_BLOQUEAR_SEGREDO` → `G_BLOQUEAR_SEGREDOS`
- **aidd-forge:** `G_CYBERSECURITY` → `G_CYBERSECURITY_OWASP`
- **aidd-factory:** `G_FACTORY_INPUT` (validação estrita do schema do plano de infraestrutura), `G_FACTORY_OUTPUT` (verificação do manifesto de entrega), `G_FACTORY_DETERMINISTIC` (garantia de zero LLM nas fases 1, 4, 5 e 6). → `G_FACTORY_ANALYSIS`, `G_FACTORY_COMPOSE`, `G_FACTORY_ENV`, `G_FACTORY_INIT_DB`, `G_FACTORY_INTEGRATION` e `G_FACTORY_MVP` (em `tools/aidd-factory/gates/`). `G_FACTORY_INPUT`, `G_FACTORY_OUTPUT` e `G_FACTORY_DETERMINISTIC` são rótulos de invariante no `AGENTS.md`, não arquivos de portão.
- **aidd-ops:** catálogo de ferramentas (`data/catalogo_ferramentas.json`) → requisitos de recursos (`data/requisitos_recursos.json`)

Epílogo: A Orquestra Perfeita

Quando você junta as **8 Ferramentas Macro**, as **68 Micro-Ferramentas**, os **50 Quality Gates** e os **141 Componentes Agnósticos**, o que você tem não é apenas um projeto de software: você tem uma **catedral digital determinística**.

No Ecossistema AIDD, nenhuma linha de código nasce sem plano, nenhuma ferramenta opera sem contrato e nenhum portão se abre sem que a prova matemática tenha sido atendida.

Fim da Auditoria Canônica — 24 de setembro de 2026.